CS330 Operating Systems and Lab.

Virtual Memory

Spring 2026

Instructor : Insik Shin

Virtual Memory

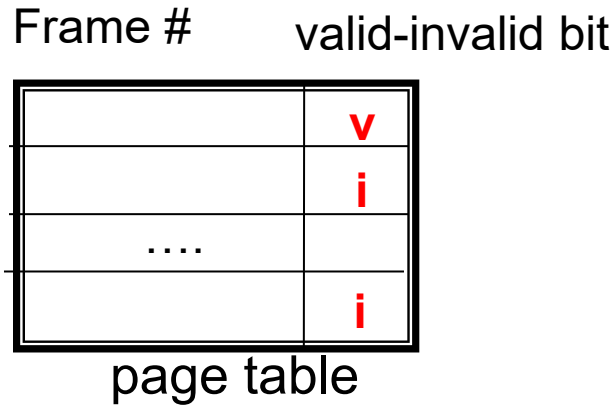
- Virtual memory
  - A technique to separate user logical memory from physical memory
  - It allows
    - Only part of the program needs to be in memory for execution
    - Logical address space can therefore be much larger than physical address space
    - Address spaces can be shared by several processes
    - More efficient process creation
- Virtual memory can be implemented via:
  - Demand paging

Demand Paging

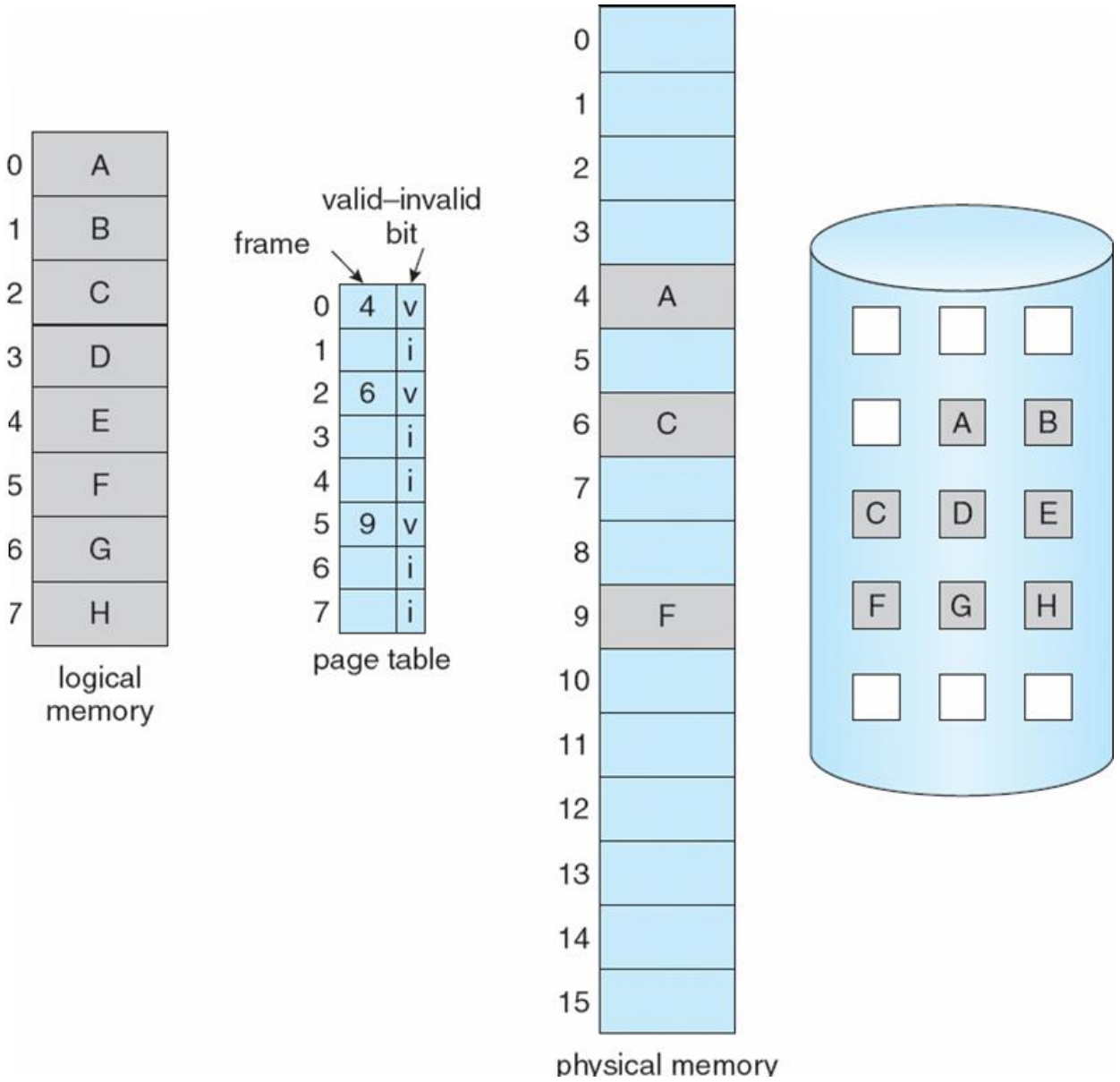
- Demand paging
  - A paging system that brings a page into memory only when it is needed
    - Less I/O needed
    - Less memory needed
    - Faster response
    - More users

Valid-Invalid Bit

- Some pages can be loaded into memory, but the others not
  - Need some form of hardware support to distinguish which pages are in memory / on disk
- Valid-invalid bit in page table
  - With each page table entry, a valid–invalid bit is associated (**v** ⇒ in-memory, **i** ⇒ not-in-memory)
  - Initially valid–invalid bit is set to **i** on all entries
  - During address translation, if valid–invalid bit in page table entry is **i** ⇒ page fault

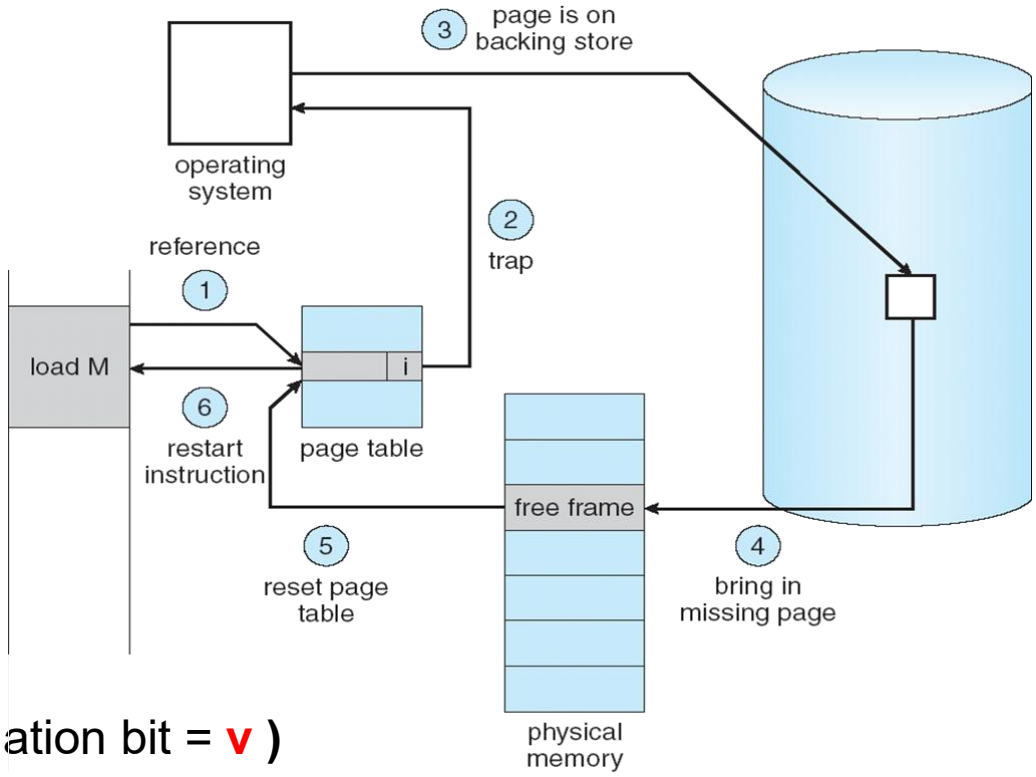


Page Table When Some Pages Are Not in Main Memory



Page Fault

- If a process tries to access a page that was not brought into memory,
  1. Paging hardware notices that the validation bit = **i** and causes a trap to OS
  2. OS looks at an internal table (in PCB) to decide:
     - Invalid reference ⇒ abort
     - Just not in memory
  3. Get empty frame



  4. Swap page into frame
  5. Reset page tables (i.e., set validation bit = **v**)
  6. Restart the instruction that caused the page fault

Performance of Demand Paging

- Page Fault Rate $0 \leq p \leq 1.0$
  - if $p = 0$, no page faults
  - if $p = 1$, every reference is a fault

- Effective Access Time (EAT)

  EAT = (1 – p) x memory access
  + p (page fault overhead + memory access)

page fault overhead = swap page out + swap page in
+ restart overhead

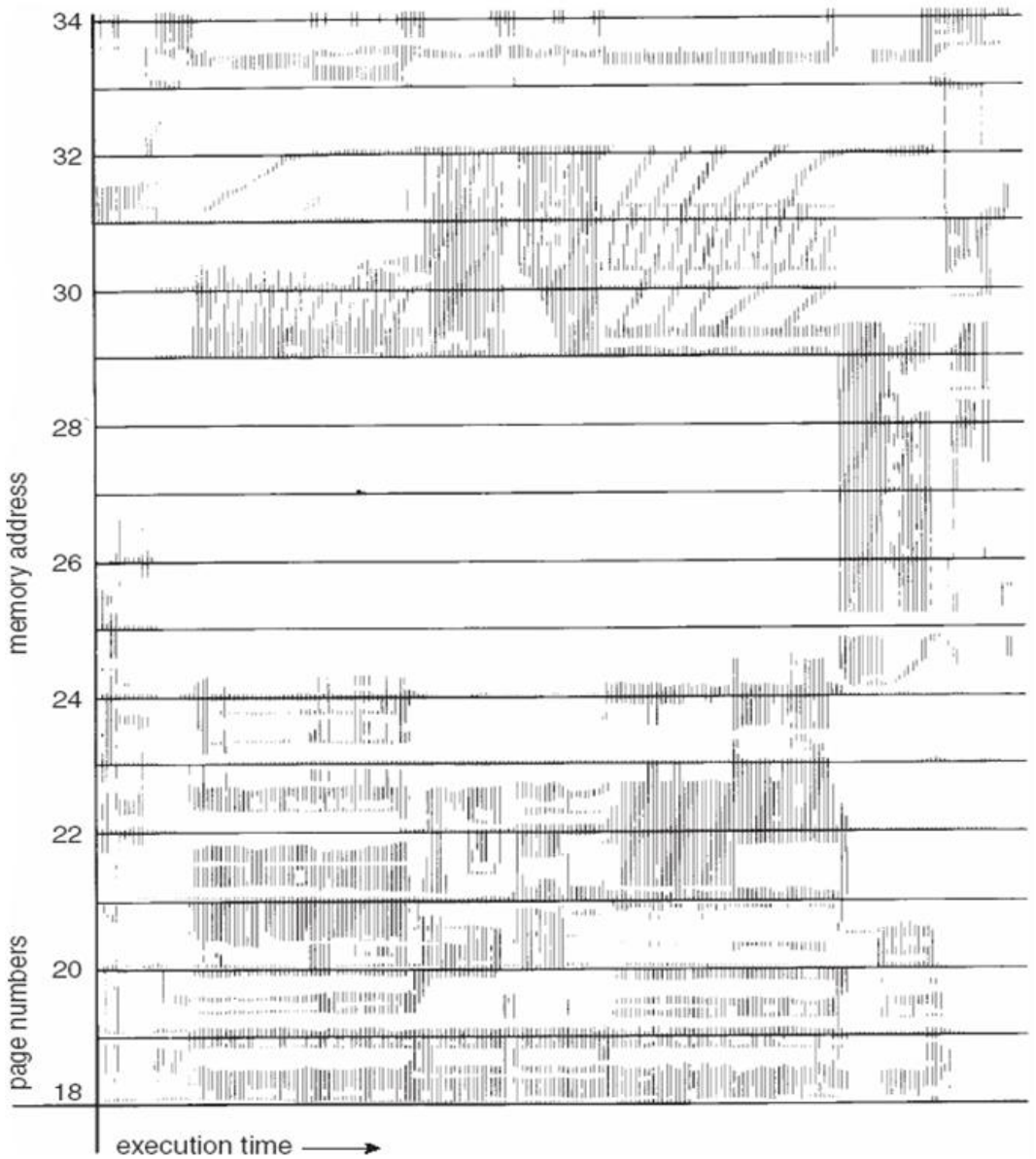
Demand Paging Example

- Memory access time = 200 nanoseconds
- Average page-fault service time = 8 milliseconds
- EAT = (1 – p) x 200 + p (8 milliseconds + 200)
  = (1 – p) x 200 + p x 8,000,200
  = 200 + p x 8,000,000
- If one access out of 1,000 causes a page fault, then EAT = 8.2 microseconds.
  This is a slowdown by a factor of 40!!

Demand Paging

- Why does this work?
  - *Locality of reference*
    - **Temporal locality**: data elements are frequently referenced again within relatively small time durations
    - **Spatial locality**: data elements within relatively close storage locations are frequently referenced
  - Locality implies infrequent paging !

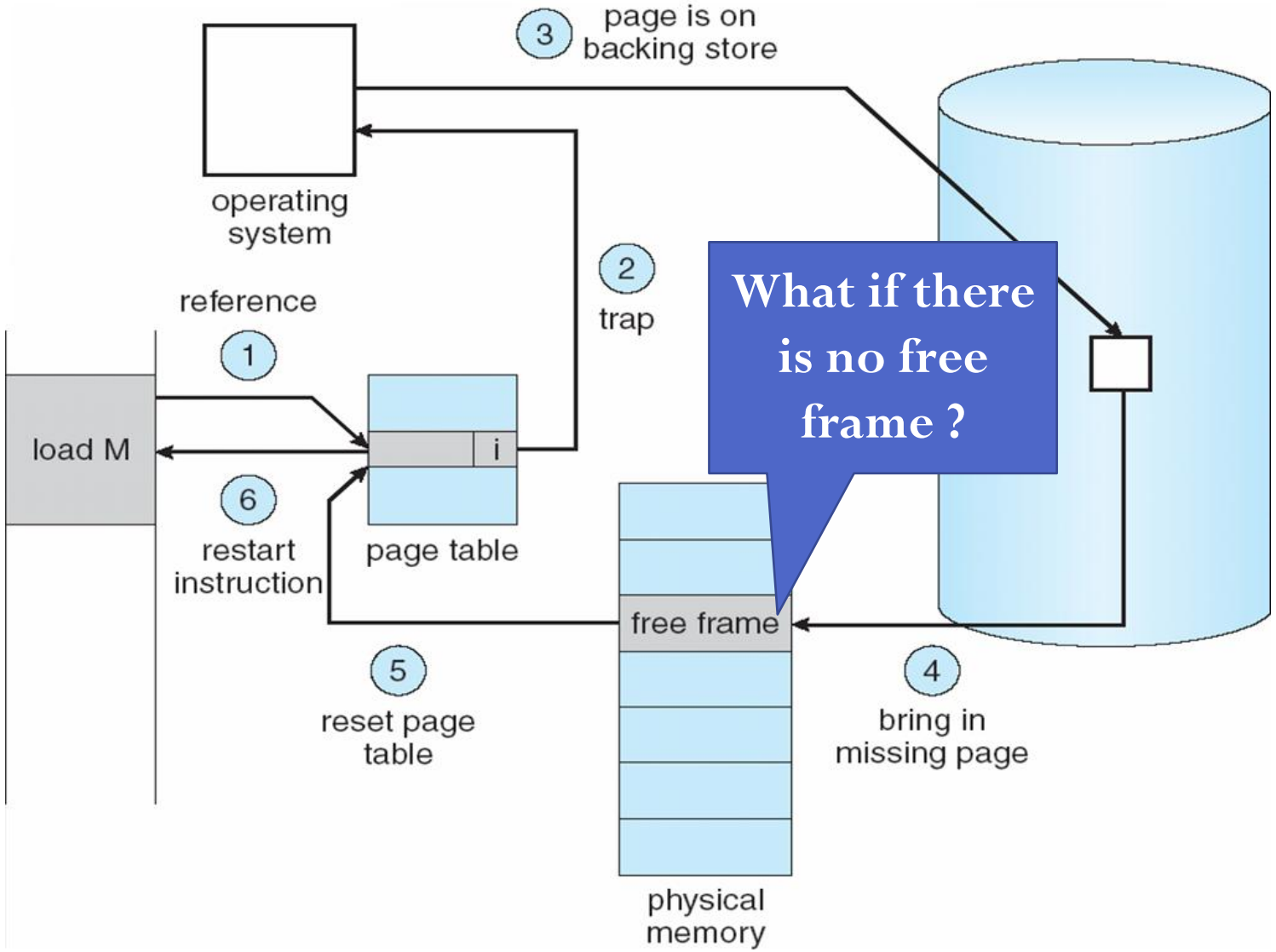
Locality In A Memory-Reference Pattern



Page Fault (revisit)

- Page fault happens if a process tries to access a page that was not brought into memory
  1. Operating system looks at another table to decide:
     - Invalid reference ⇒ abort
     - Just not in memory ⇒ trap
  2. Get empty frame
  3. Swap page into frame
  4. Reset page tables (i.e., set validation bit = **v**)
  5. Restart the instruction that caused the page fault

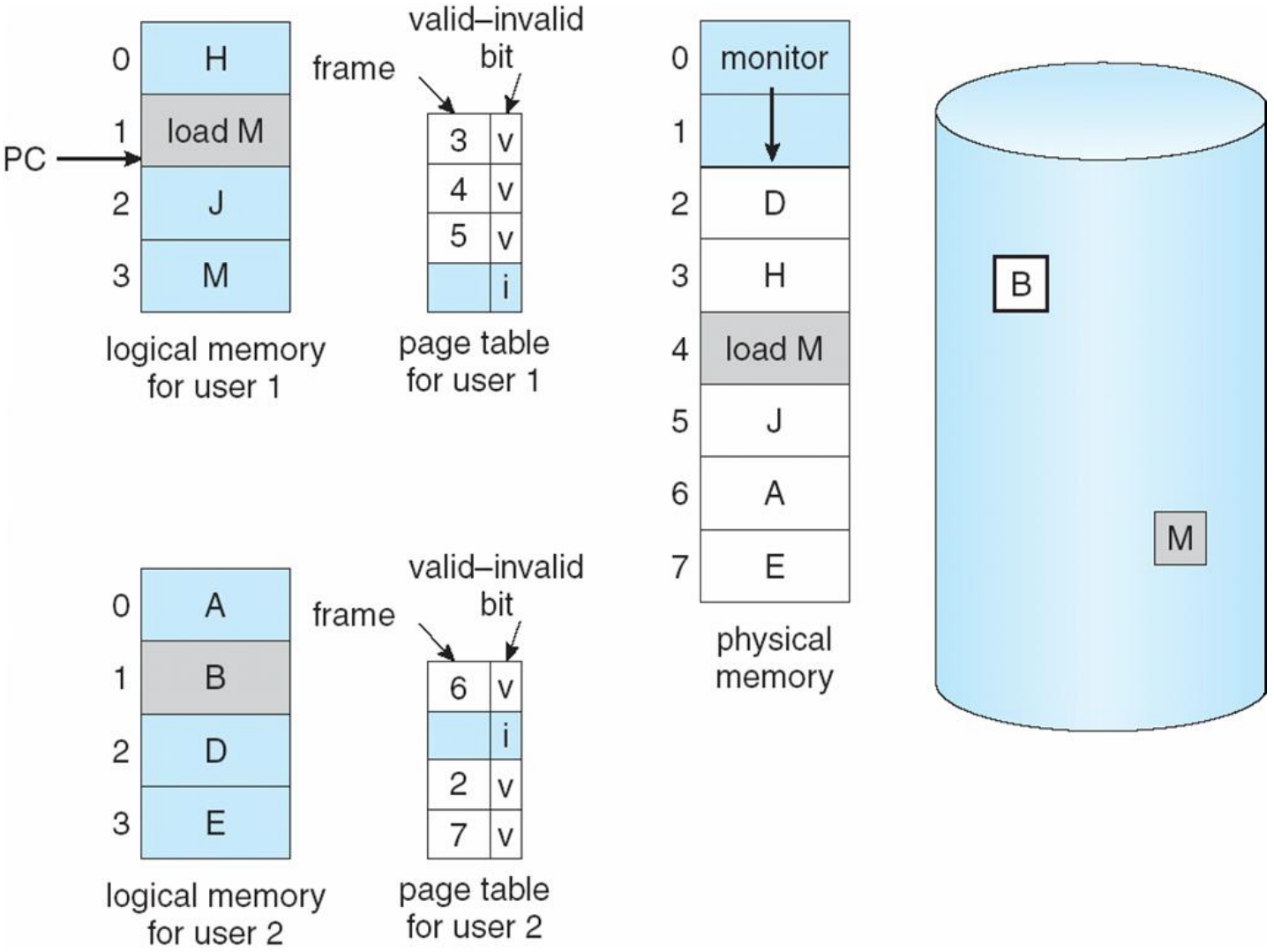
Steps in Handling a Page Fault



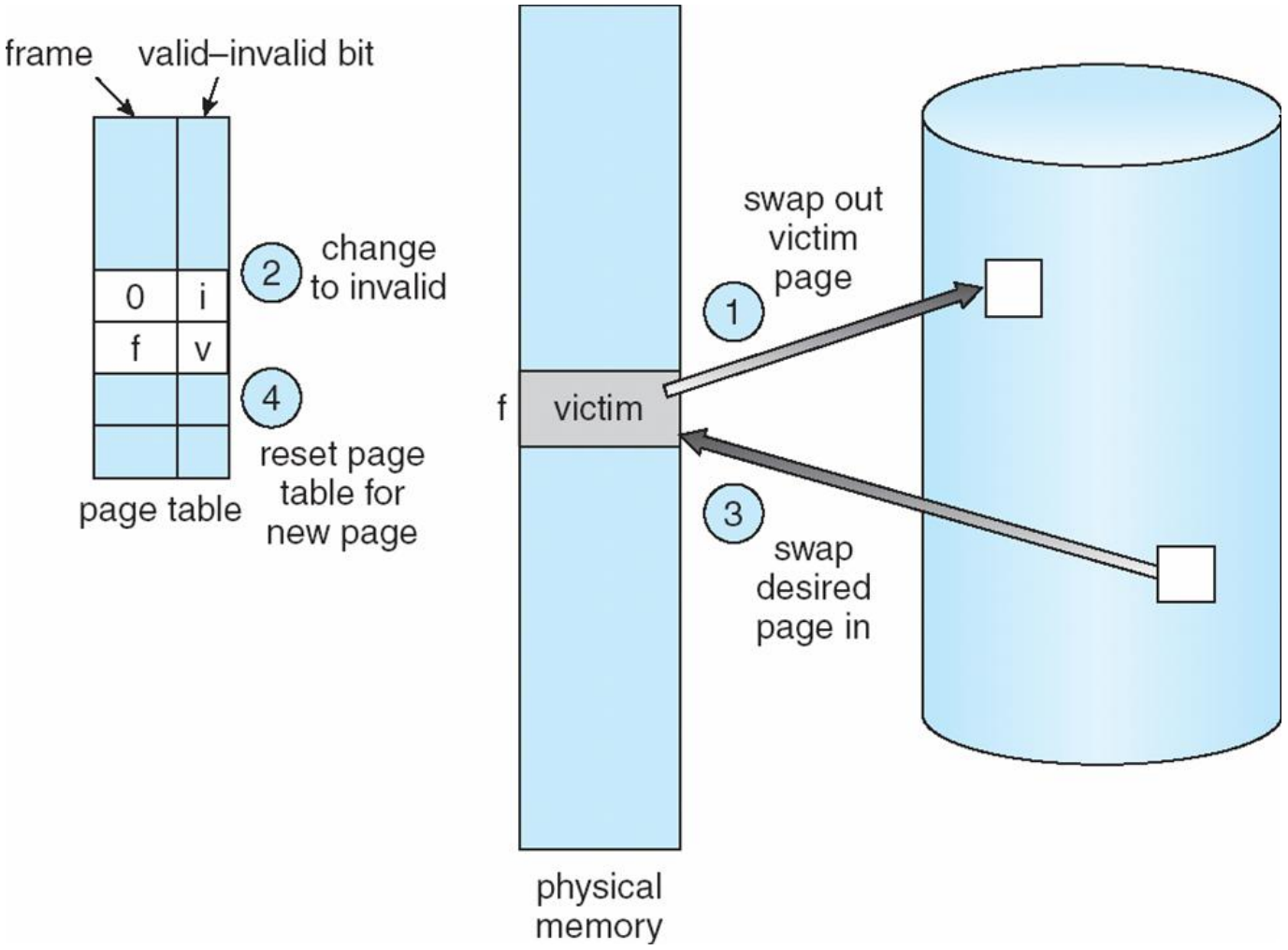
What happens if there is no free frame?

- Page replacement – find some page in memory, but not really in use, swap it out
  - algorithm
  - performance – want an algorithm which will result in minimum number of page faults
- Same pages may be brought into memory several times

Need For Page Replacement



Page Replacement



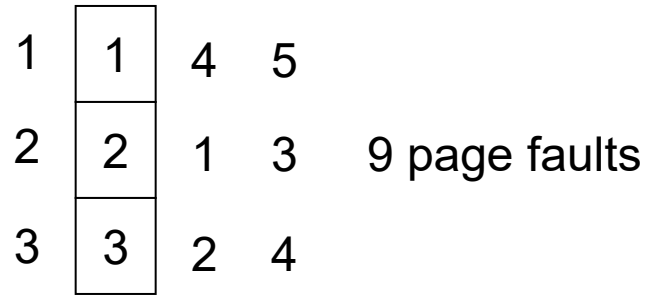
Page Replacement Algorithms

- Select a victim page to swap out, aiming at a lower page-fault rate
- Evaluate algorithm by running it on a particular string of memory references (reference string) and computing the number of page faults on that string
- In all our examples, the reference string is

1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

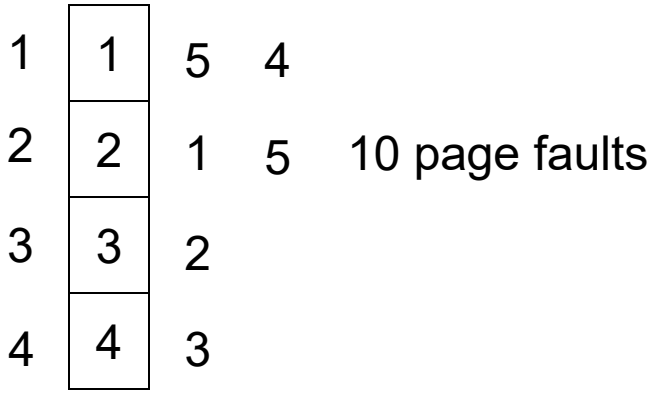
First-In-First-Out (FIFO) Algorithm

- Reference string: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5
- 3 frames (3 pages can be in memory at a time per process)



9 page faults

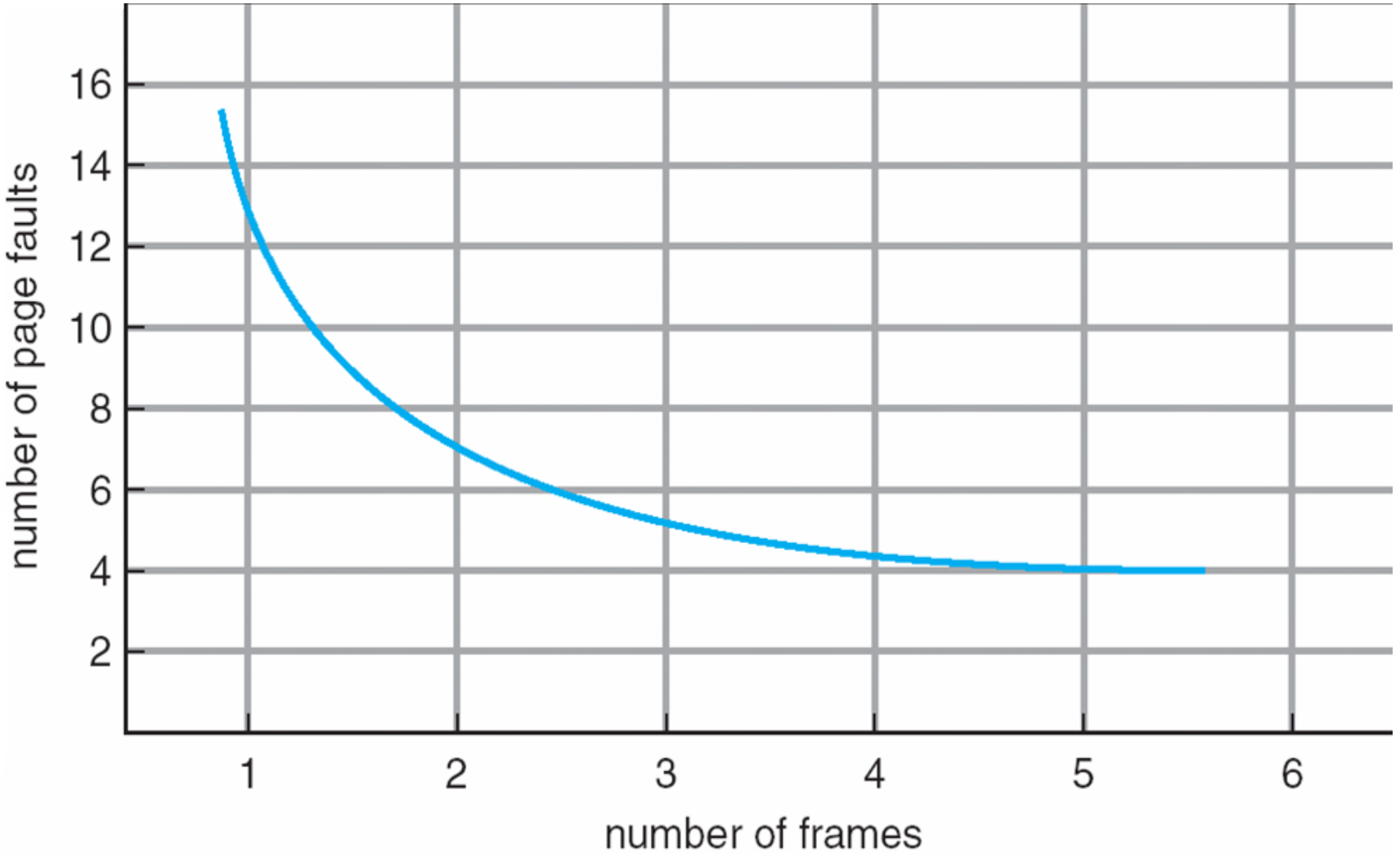
- 4 frames



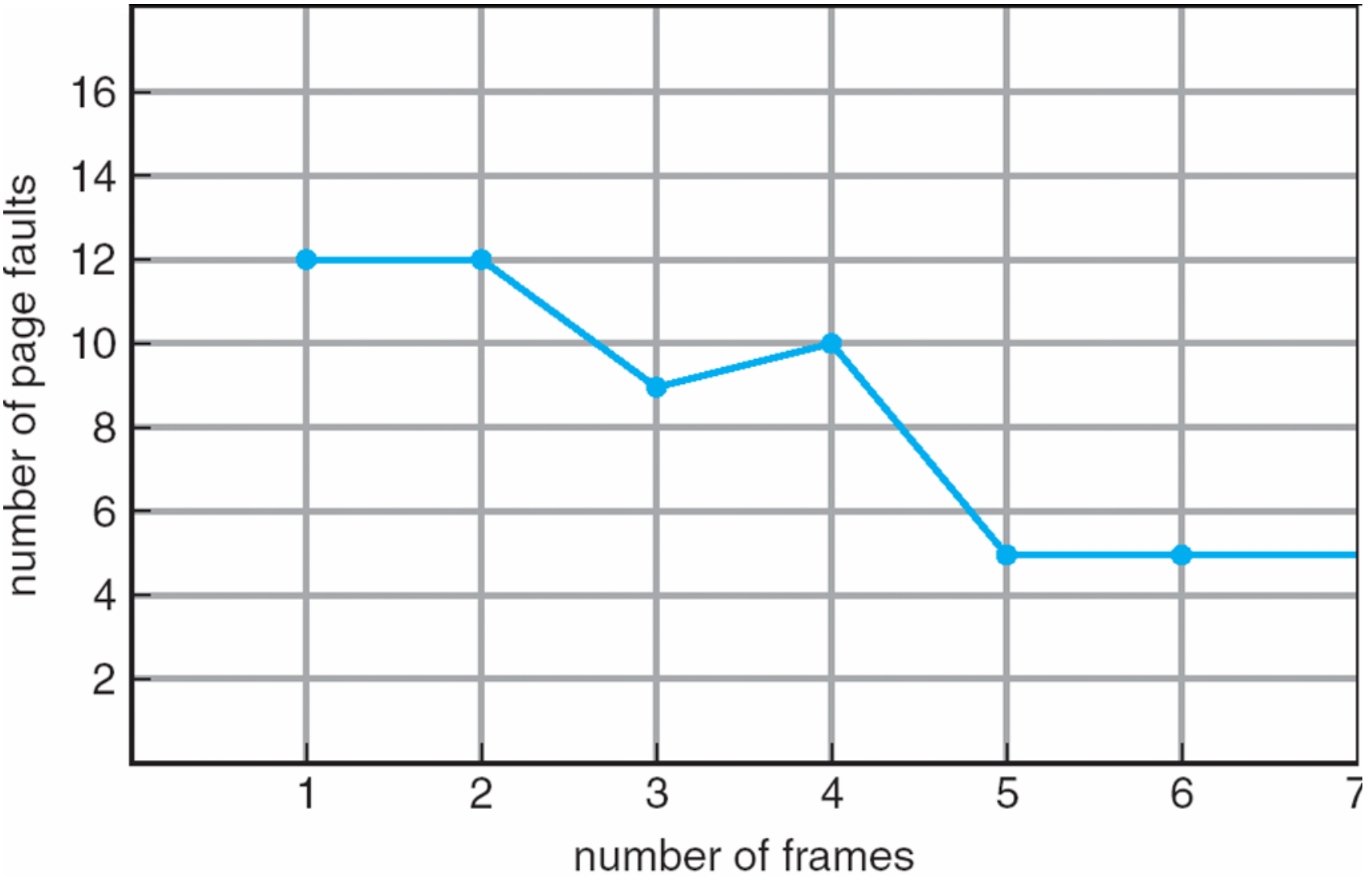
10 page faults

- Belady's Anomaly: more frames ⇒ more page faults

Graph of Page Faults vs. the Number of Frames



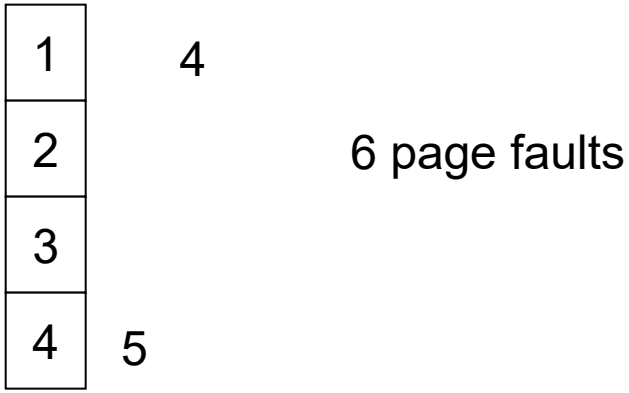
FIFO Illustrating Belady's Anomaly



Optimal Algorithm

- Replace page that ***will not be used*** for longest period of time
- 4 frames example

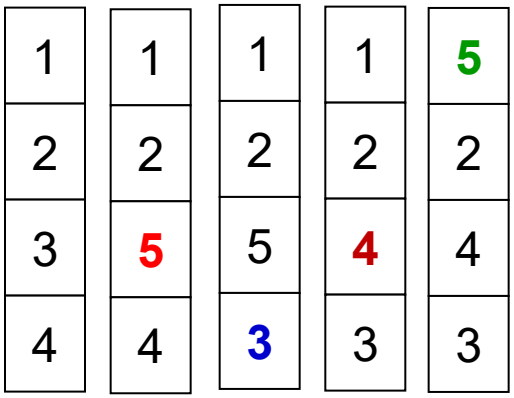
1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5



- How do you know this?
- Used for measuring how well your algorithm performs

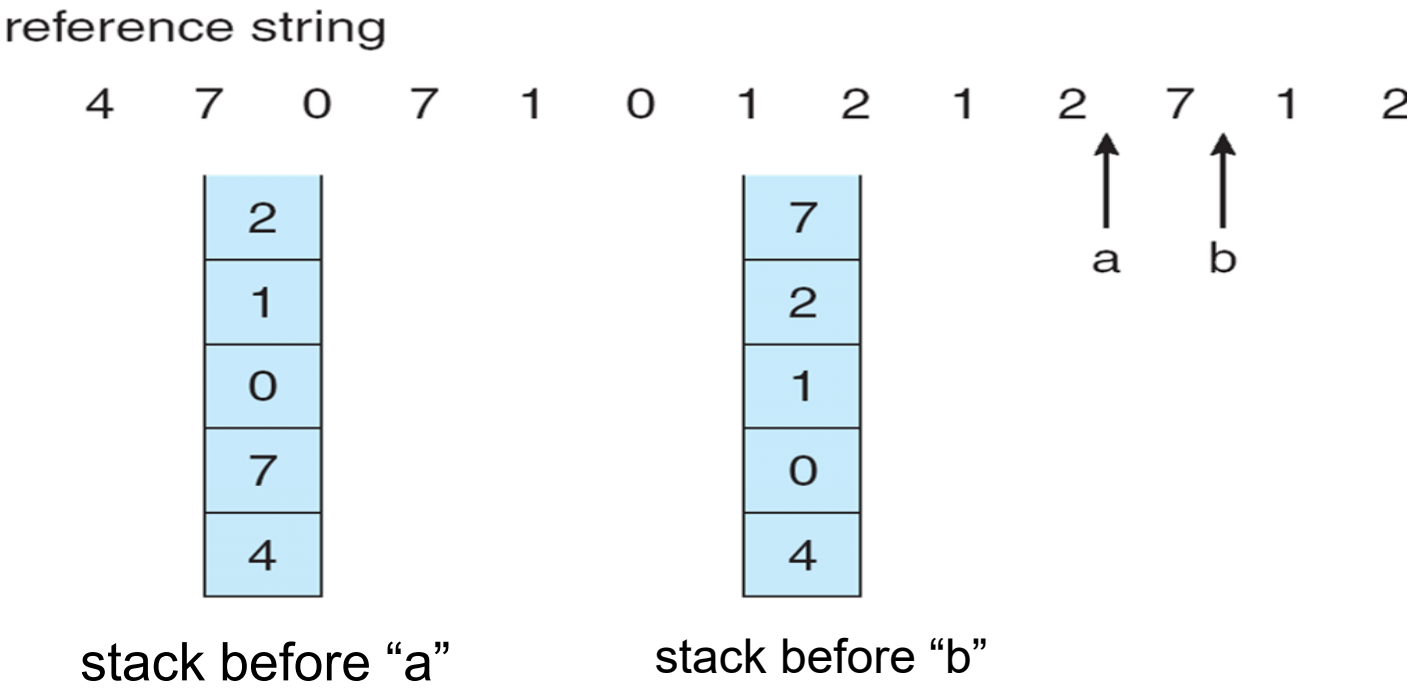
Least Recently Used (LRU) Algorithm

- Replace the page that ***has not been used*** for the longest time
- Reference string: 1, 2, 3, 4, 1, 2, **5**, 1, 2, **3**, **4**, **5**

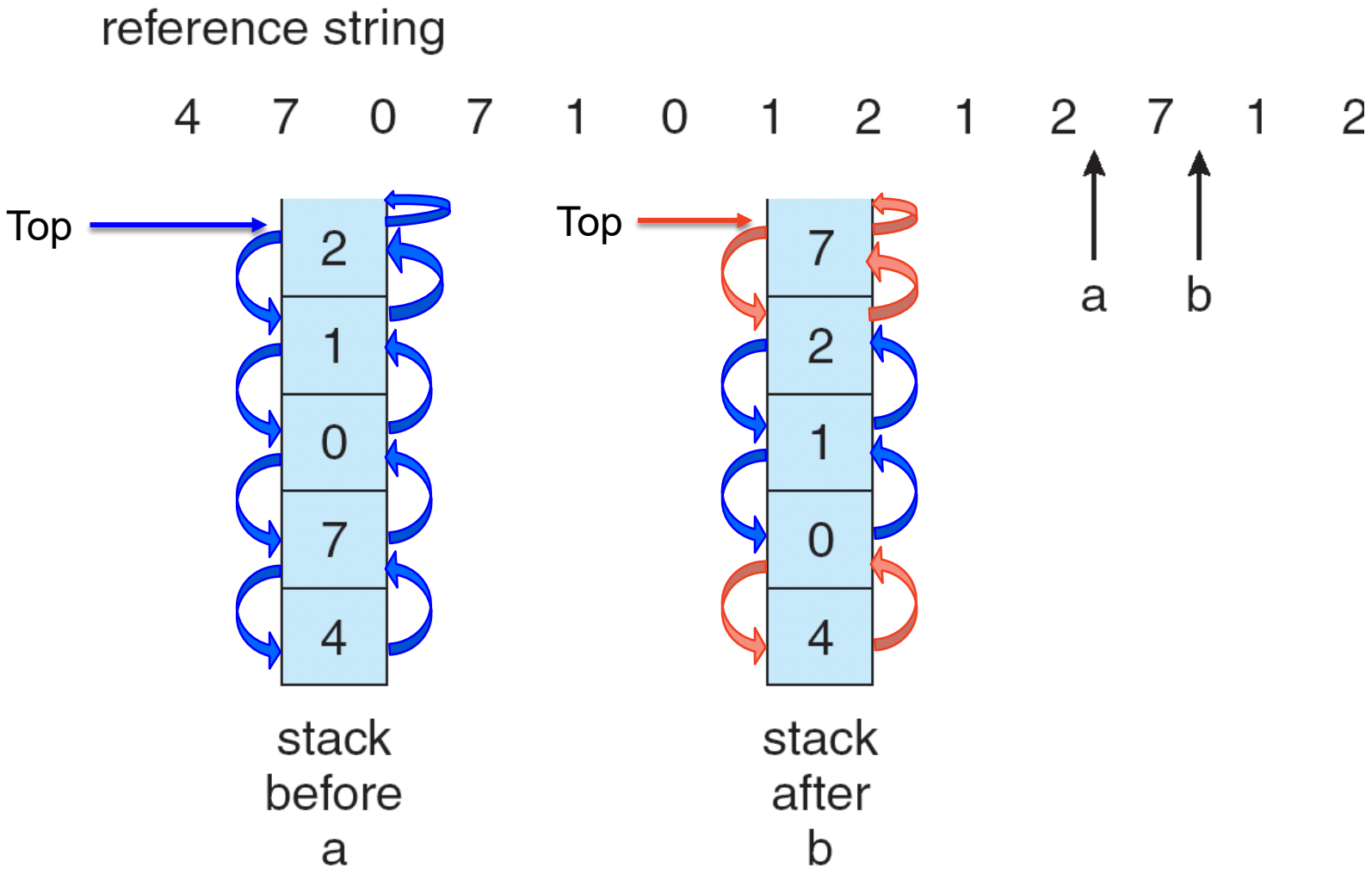


LRU Algorithm (Cont.)

- Stack implementation – keep a stack of page numbers in a double link form:
  - Page referenced:
    - move it to the top; requires 6 pointers to be changed
  - No search for replacement

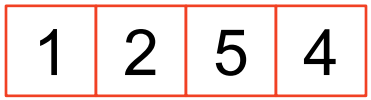


Use Of A Stack to Record The Most Recent Page References



Stack Algorithms

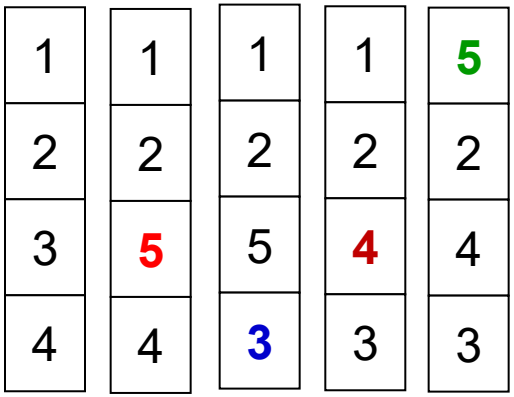
- Interesting feature
  - Stack algorithms, such as optimal and LRU, can never exhibit Belady's anomaly
- Stack algorithm
  - A set of pages in memory for *n* frames is always a *subset* of the set of pages in memory for *n*+1 frames
    - *4* frames



    - *5* frames

Least Recently Used (LRU) Algorithm

- Replace the page that *has not been used* for the longest time
- Reference string: 1, 2, 3, 4, 1, 2, **5**, 1, 2, **3**, **4**, **5**



- Counter/clock implementation
  - Every page entry has a counter; every time page is referenced through this entry, copy the clock into the counter
  - When a page needs to be changed, look at the counters to determine which are to change

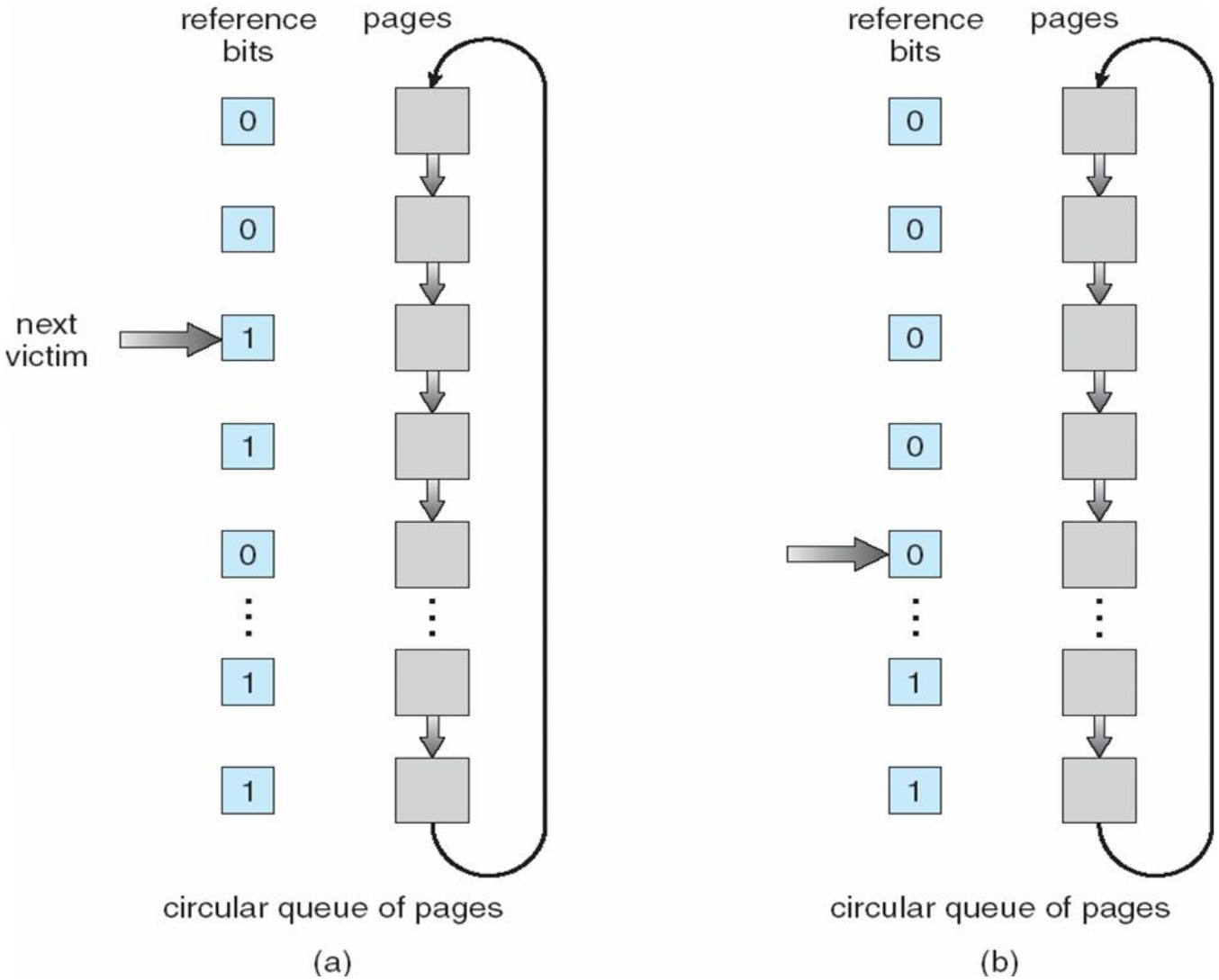
LRU-Approximation Algorithms

- Background
  - Little hardware support for true LRU, but some h/w help available for LRU approximation
- Additional-reference-bits algorithm
  - Take advantage of the reference bit of PTE
  - At regular intervals, record the reference bit for each page to figure out later which page has been used less recently

LRU-Approximation Algorithms

- Second-chance algorithm
  - Need reference bit
  - Clock replacement (or FIFO replacement)
  - If page to be replaced (in clock order) has reference bit = 1 then:
    - set reference bit 0
    - leave page in memory
    - replace next page (in clock order), subject to same rules

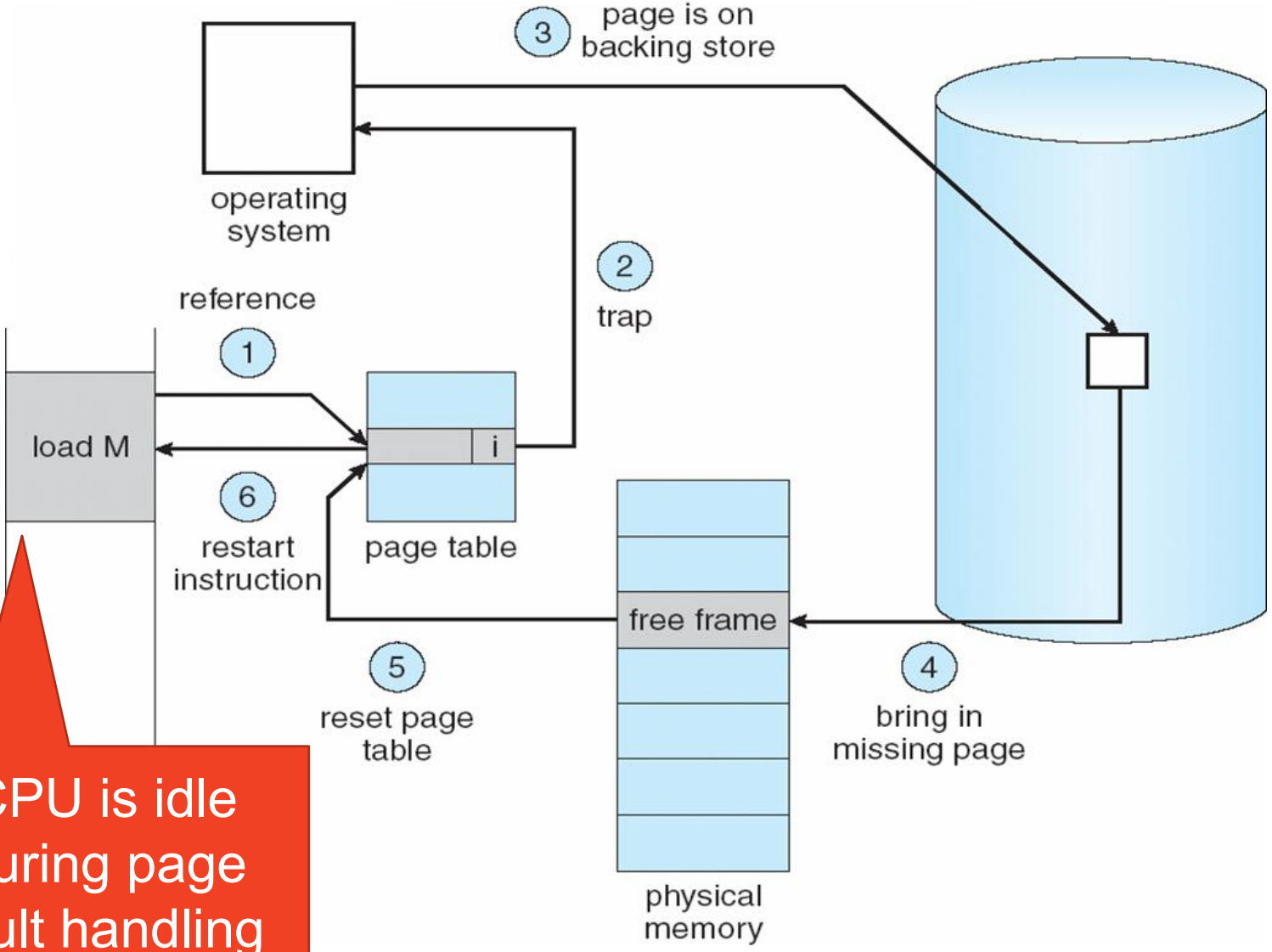
Second-Chance (clock) Page-Replacement Algorithm



Counting Algorithms

- Keep a counter of the number of references that have been made to each page
- **LFU Algorithm**: replaces page with smallest count
- **MFU Algorithm**: based on the argument that the page with the smallest count was probably just brought in and has yet to be used

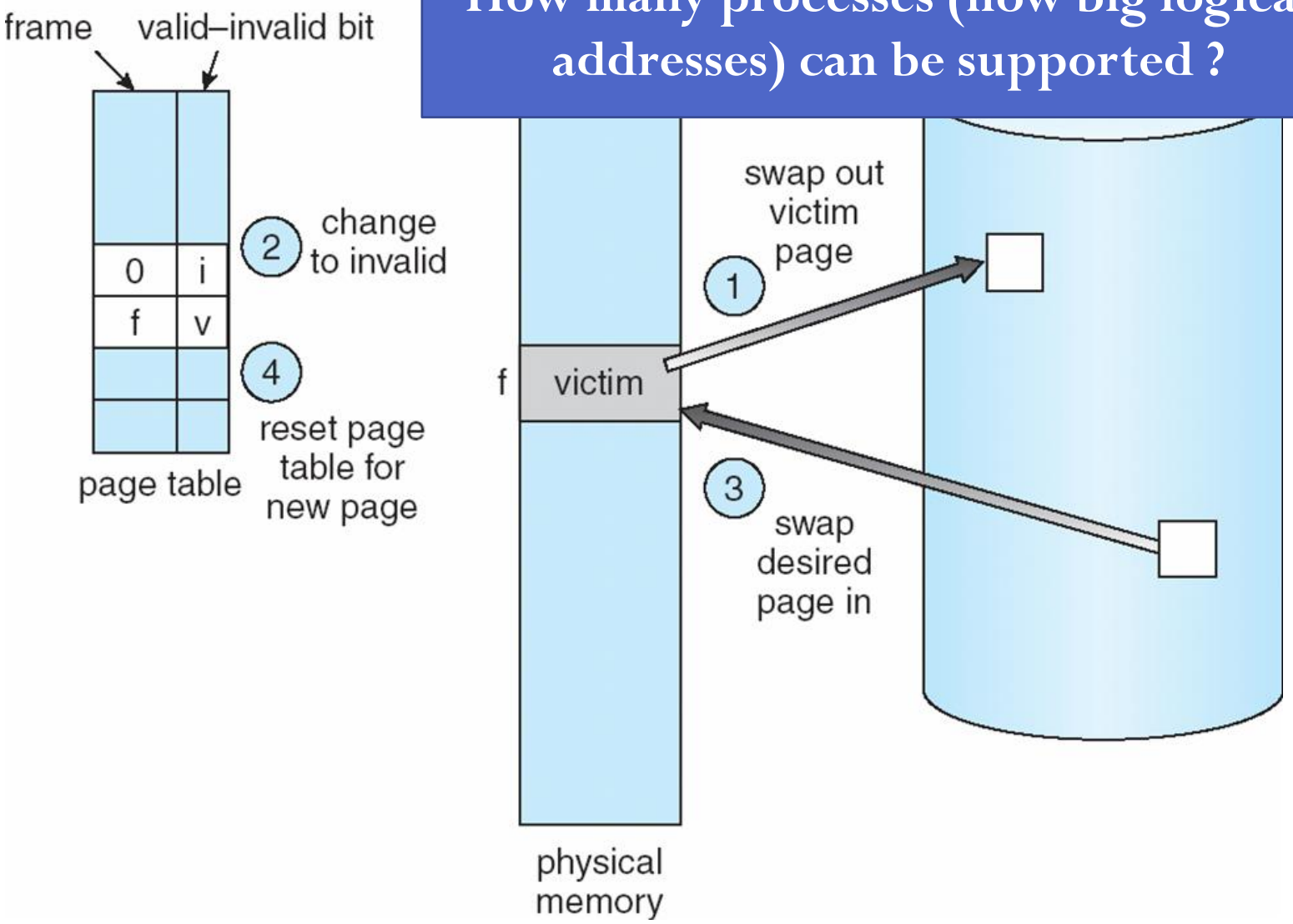
Steps in Handling a Page Fault



CPU is idle during page fault handling

Page Replacement (Memory Reclamation)

How many processes (how big logical addresses) can be supported ?

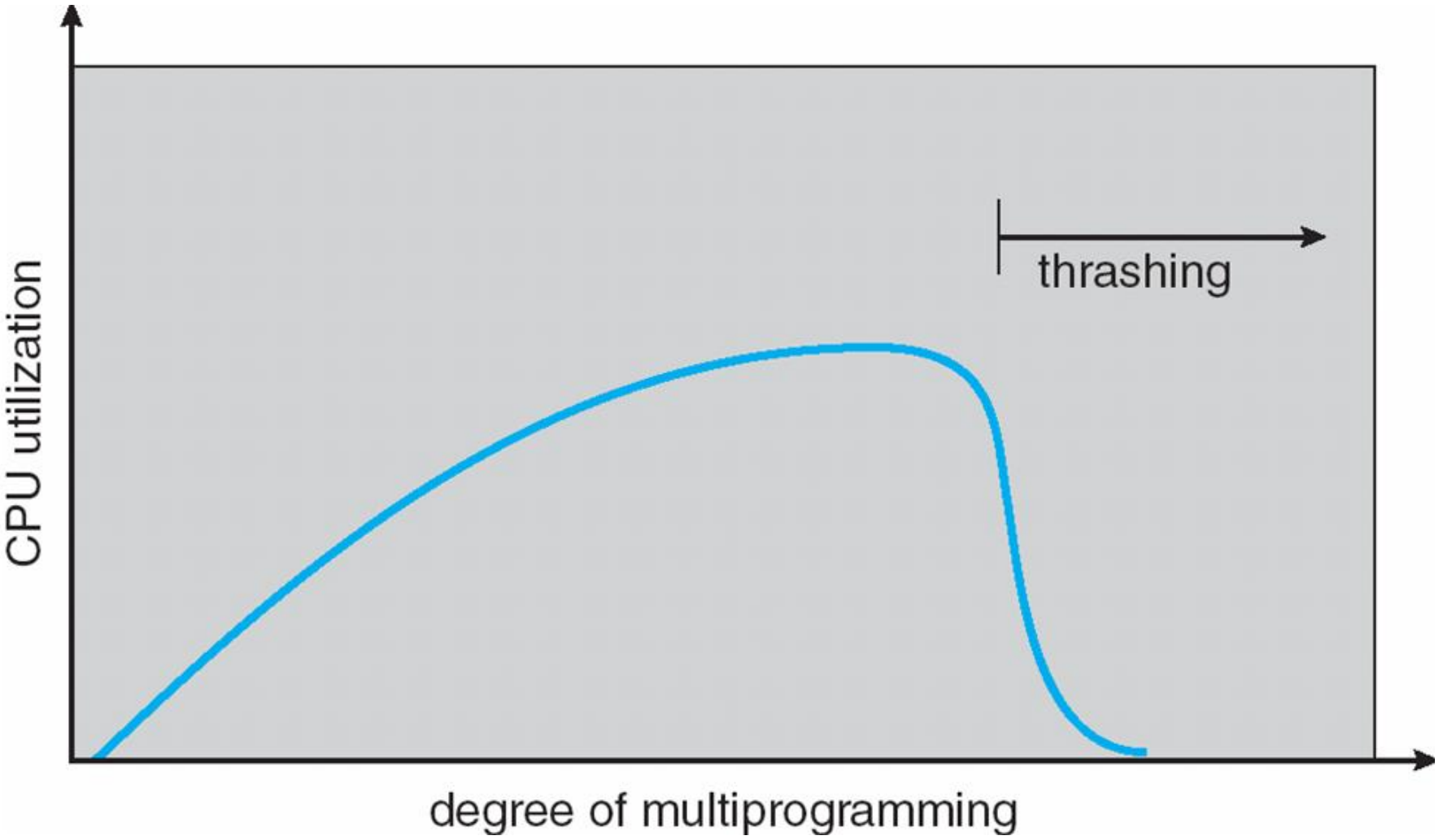


Thrashing

- If a process does not have “enough” pages, the page-fault rate is very high. This leads to:
  - low CPU utilization
  - operating system thinks that it needs to increase the degree of multiprogramming
  - another process added to the system

- **Thrashing** ≡ a process is spending much time in paging (maybe more time than executing)

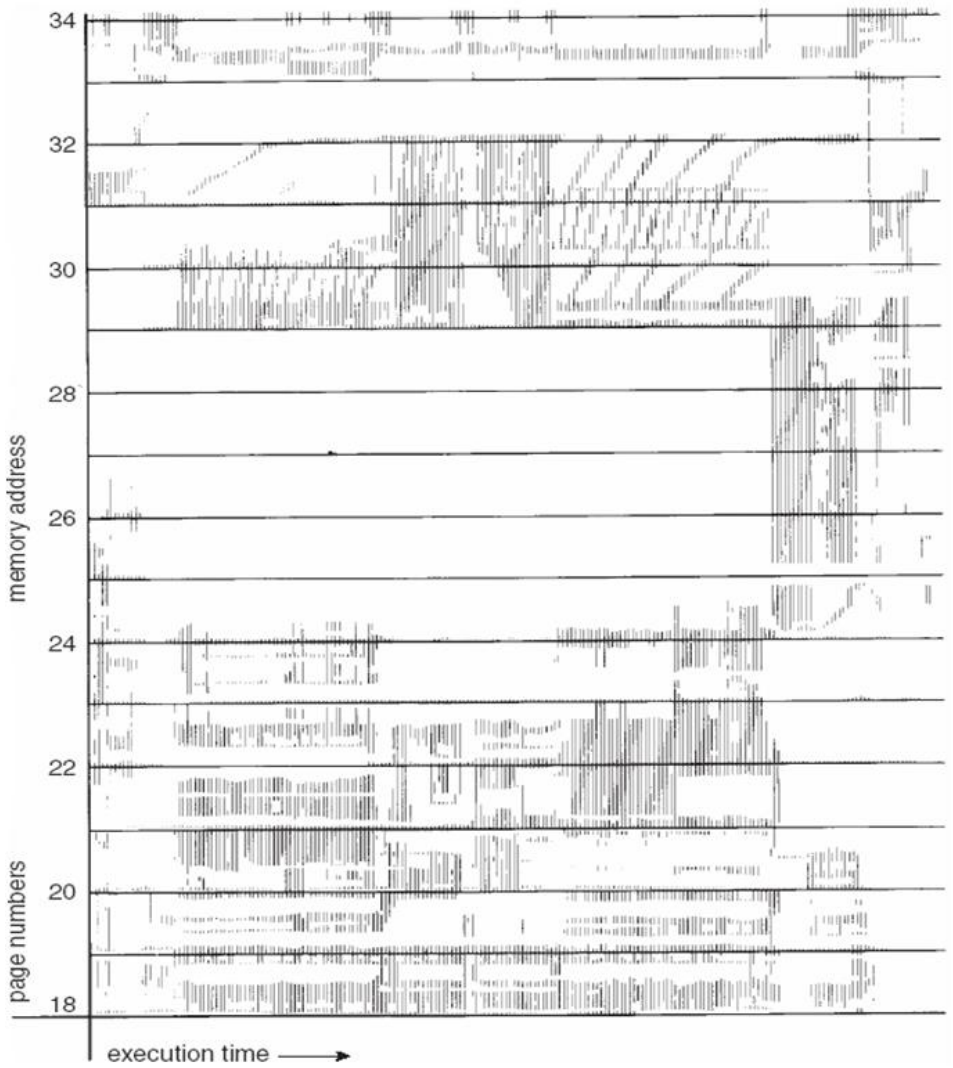
Thrashing (Cont.)



Demand Paging and Thrashing

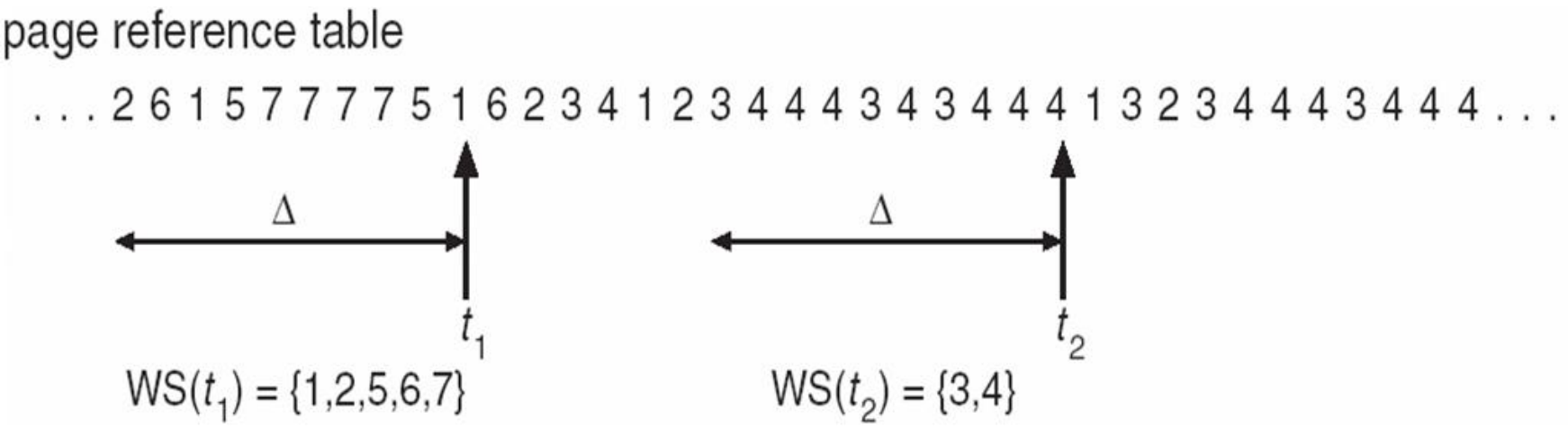
- Why does demand paging work?
  - Locality model
    - Process migrates from one locality to another
    - Localities may overlap
- Why does thrashing occur?
  - Σ size of locality > total memory size

Locality In A Memory-Reference Pattern



Working Set Model

- Working set window (Δ)
  - a fixed number of page references (i.e., 10,000 instruction)
- Working set
  - A set of pages that a process references in the last Δ
  - Approximation of program's locality
- Working set size
  - A total number of pages in the working set (varies in time)



Working Set Model

- Working set window (Δ)
  - if Δ is too small, it will not encompass entire locality
  - if Δ is too large, it will encompass several localities
  - if is $\Delta = \infty$, then it will encompass entire program

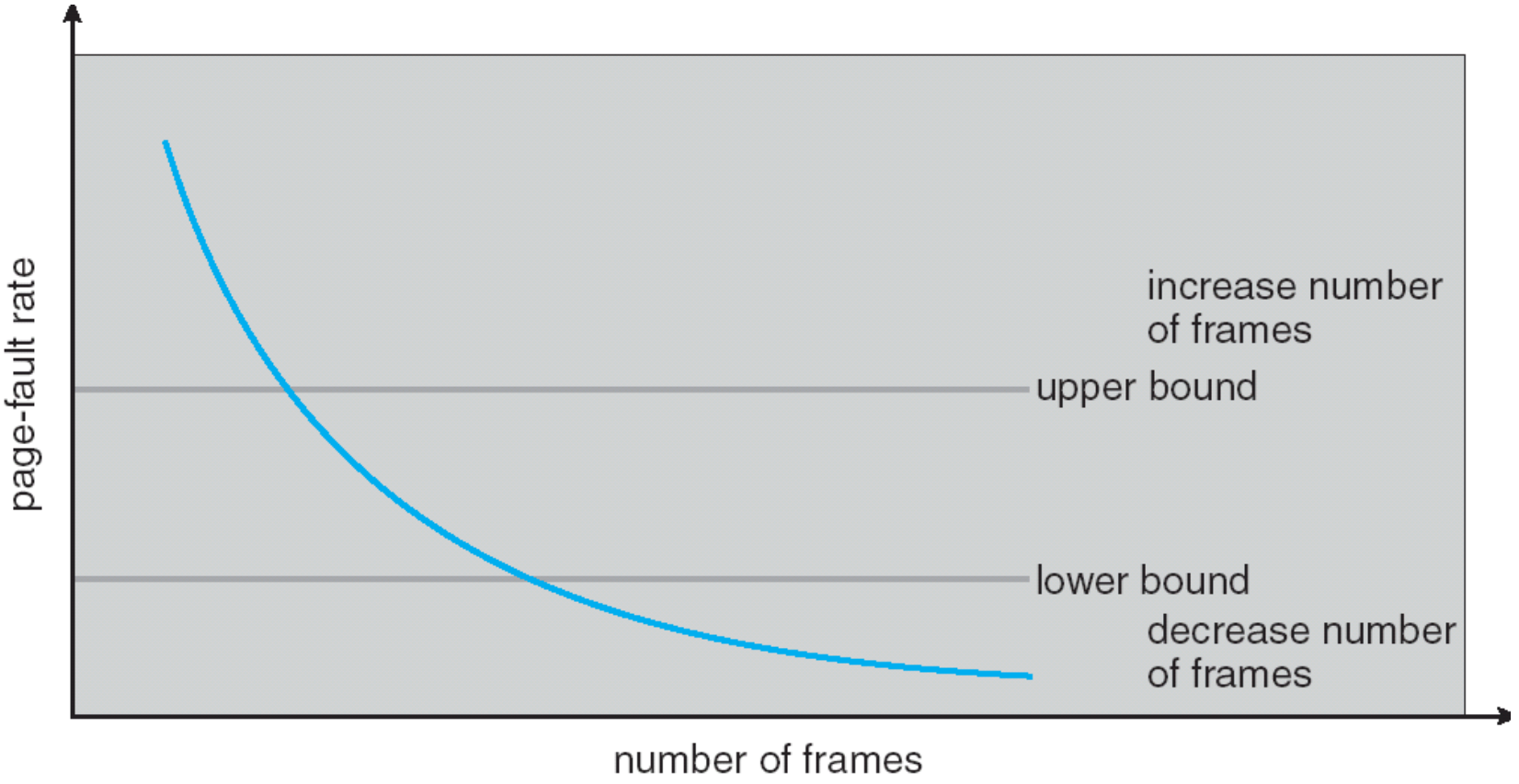
- Working-set strategy can prevent thrashing
  - $D = \Sigma\ WSS_i \equiv$ total demand frames
  - if $D > m \Rightarrow$ Thrashing (m: total number of available frames)
  - Policy if $D >$ m, then suspend one of the processes

Page-Fault Frequency Scheme

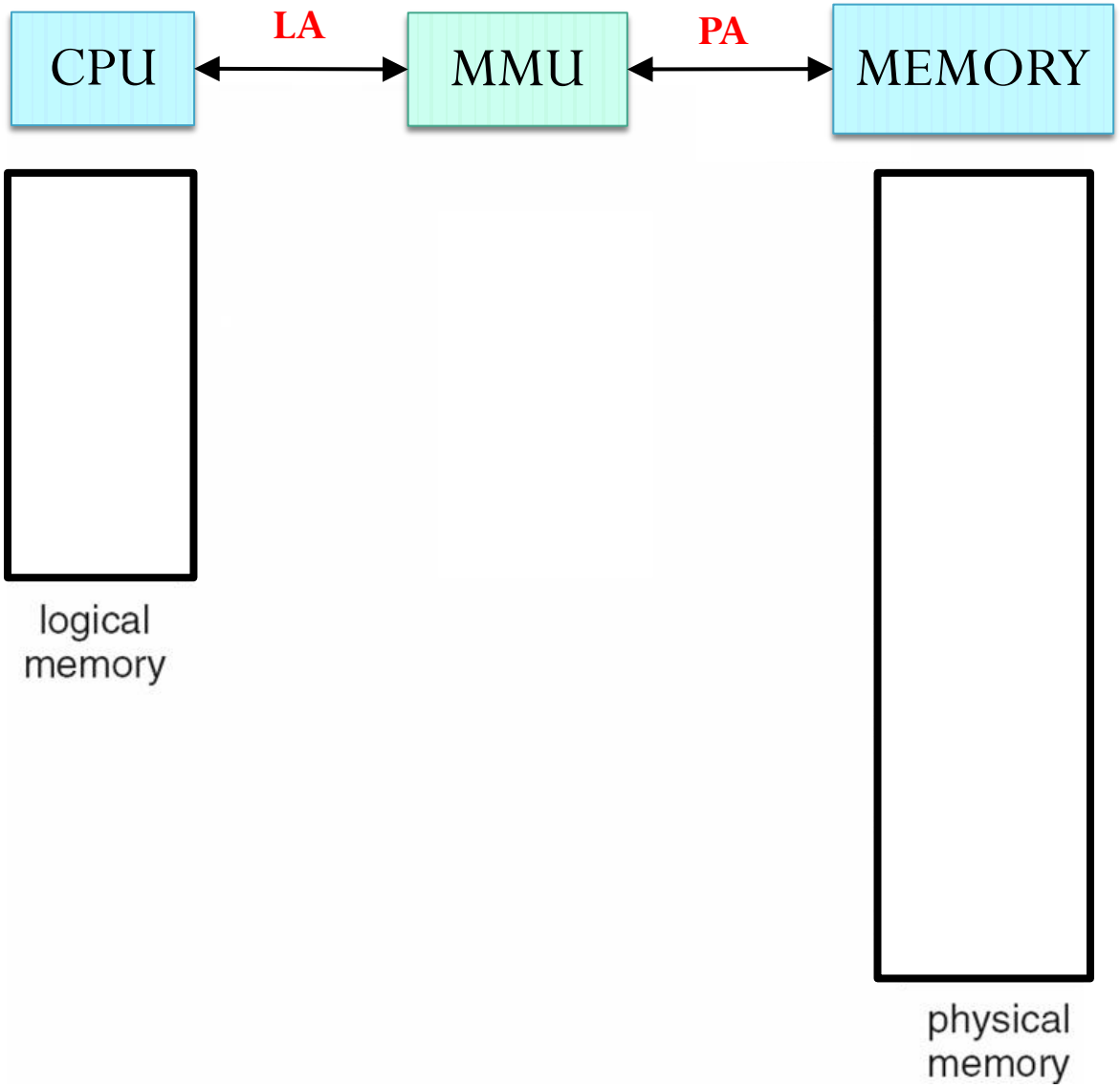
- Thrashing handling
  - Needs to detect thrashing
    - Working-set model works ($D > m$); but clumsy
    - Page-fault frequency scheme is more direct
- Page fault frequency scheme
  - Thrashing has a high page-fault rate

Page-Fault Frequency Scheme

- Establish "acceptable" page-fault rate
  - If actual rate too low, process loses frame
  - If actual rate too high, process gains frame



Paging Model of Logical and Physical Memory



Process Creation

- Virtual memory allows other benefits during process creation:

  - Memory Sharing

  - Copy-on-Write

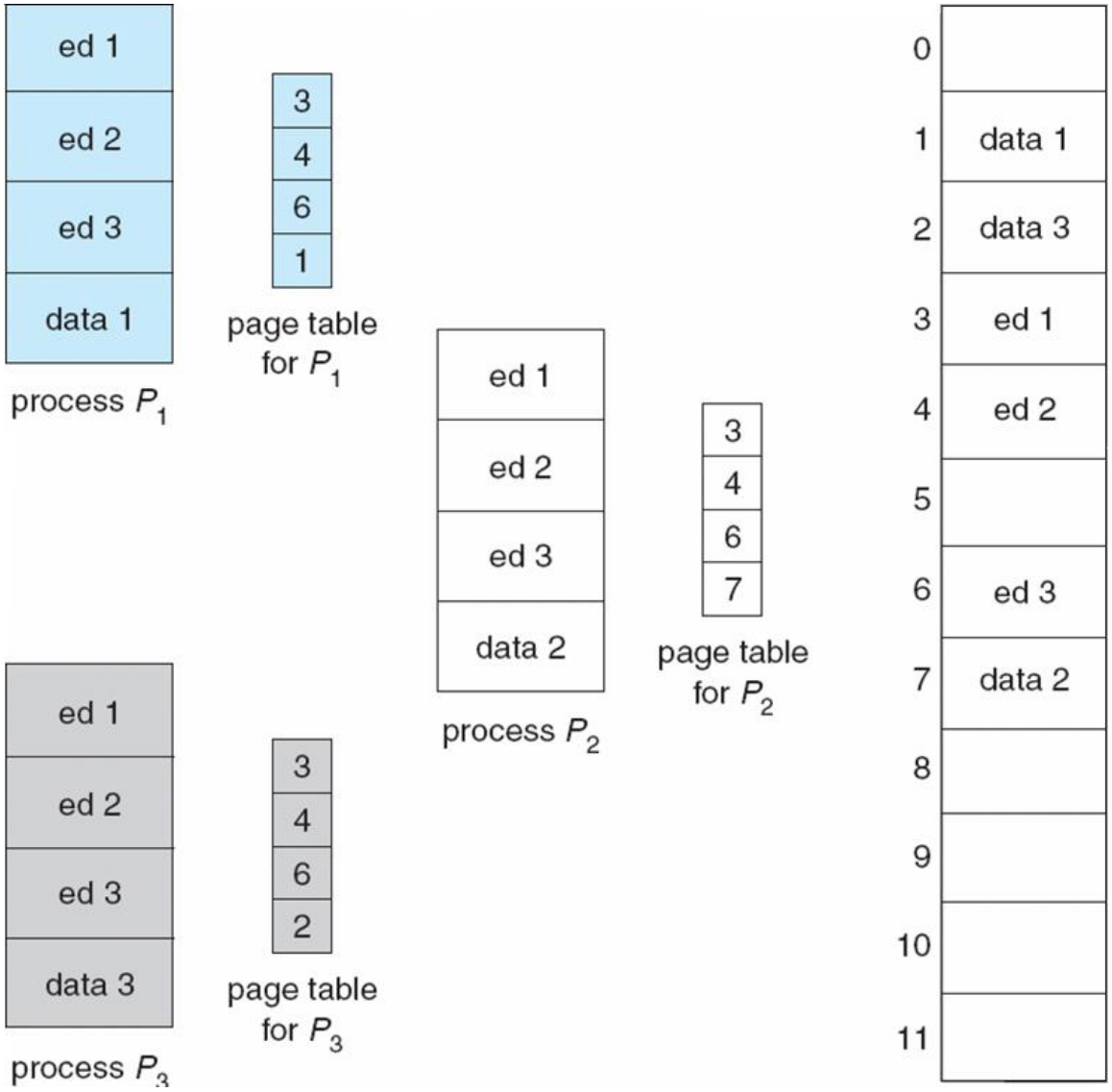
  - Memory-Mapped Files

Page Sharing

- Private virtual address space protect applications from each other
  - However, this makes it difficult to share data

- Paging makes sharing easier
  - Multiple processes can have their own logical addresses mapped to the same physical address
    - One copy of read-only (reentrant) code shared among processes (i.e., text editors, compilers, window systems).

Shared Pages Example



Copy-on-Write

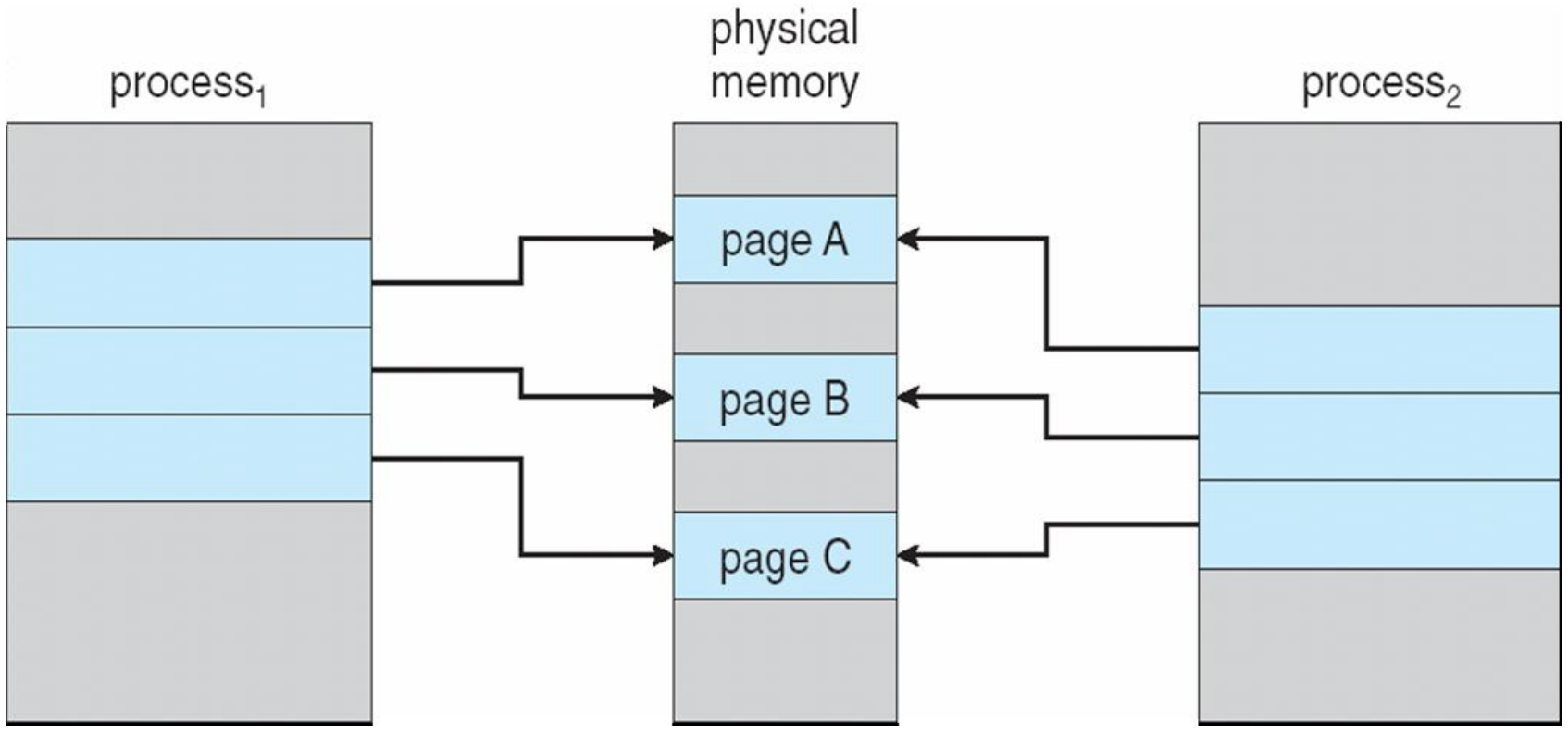
- Copy-on-Write (COW) allows both parent and child processes to initially *share* the same pages in memory

  If either process modifies a shared page, only then is the page copied

- COW allows more efficient process creation as only modified pages are copied

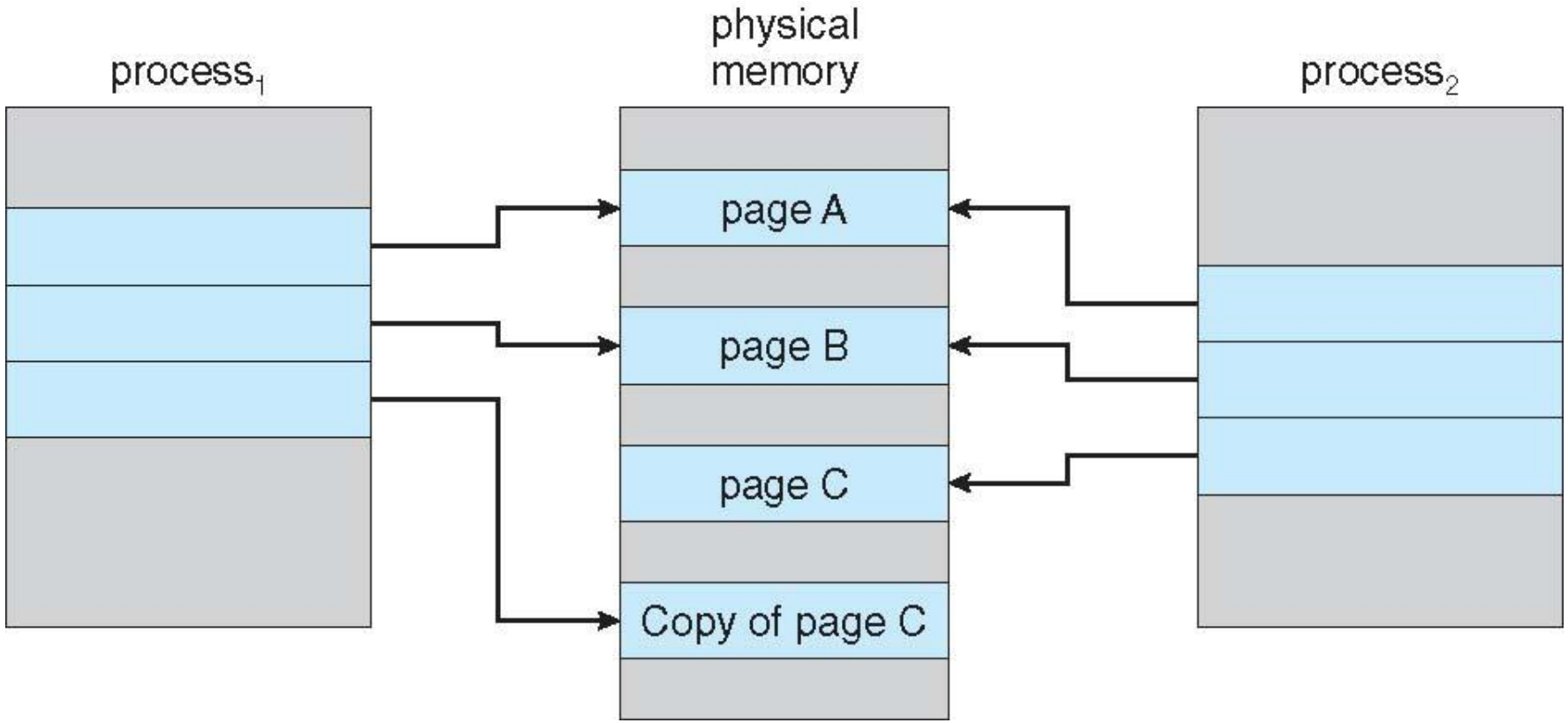
- Free pages are allocated from a **pool** of zeroed-out pages

Before Process 1 Modifies Page C



Suppose Process 1 wants to write something on page C

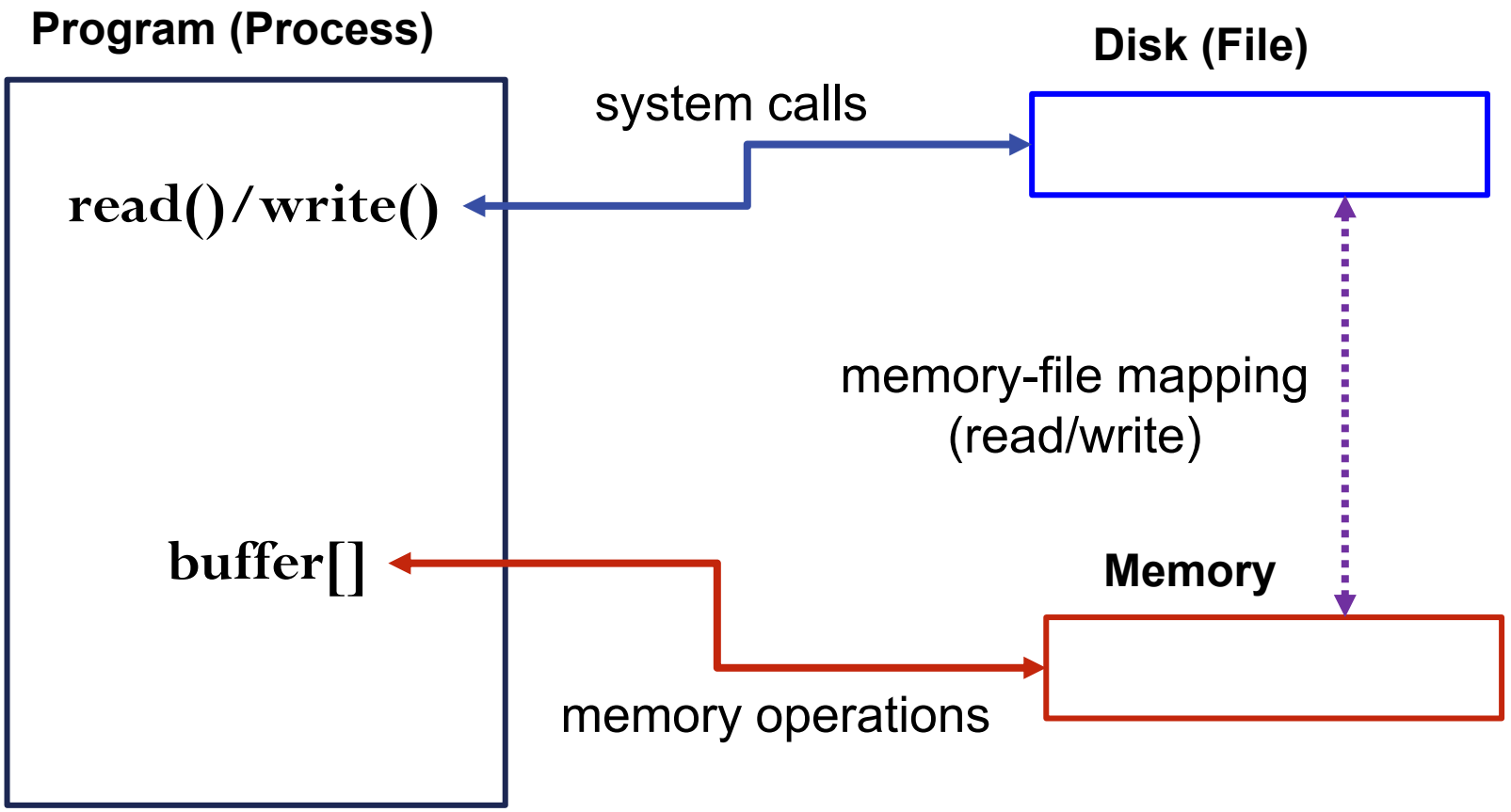
After Process 1 Modifies Page C



Memory-Mapped Files

- Consider a user program that reads a long file
  - as examples, editing documents, watching videos, etc.
- Two ways to read/write files
  - through **read()/write()** systems calls
  - through **memory-mapped files** (memory operations)
- Performance issues
  - system call vs. memory operations

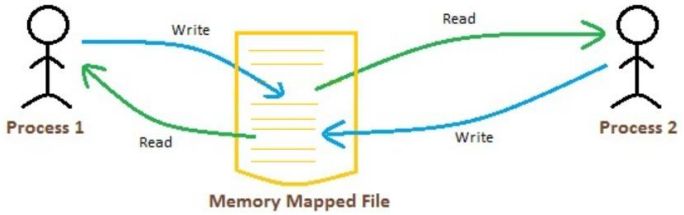
Memory-Mapped Files

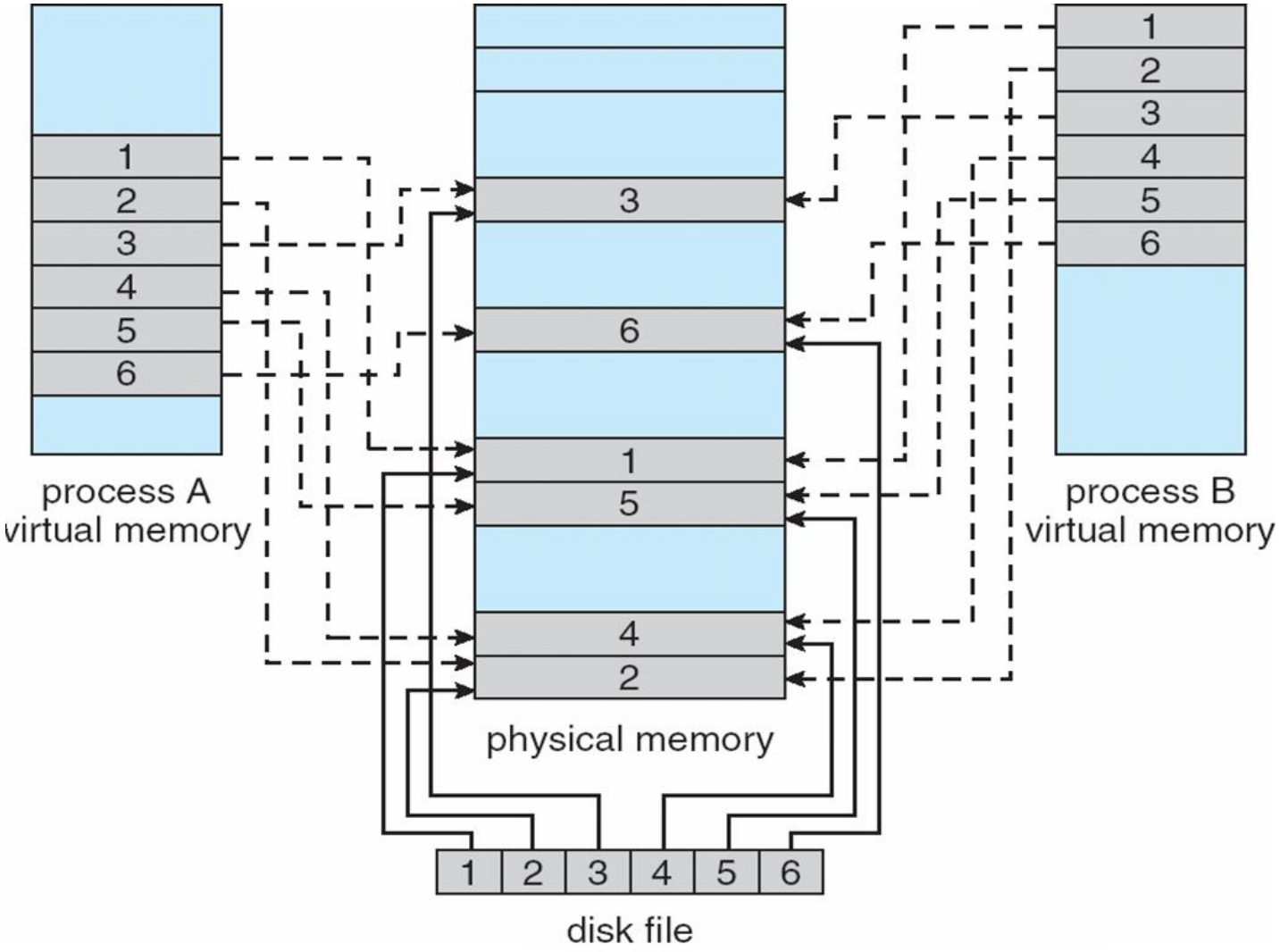


Memory-Mapped Files

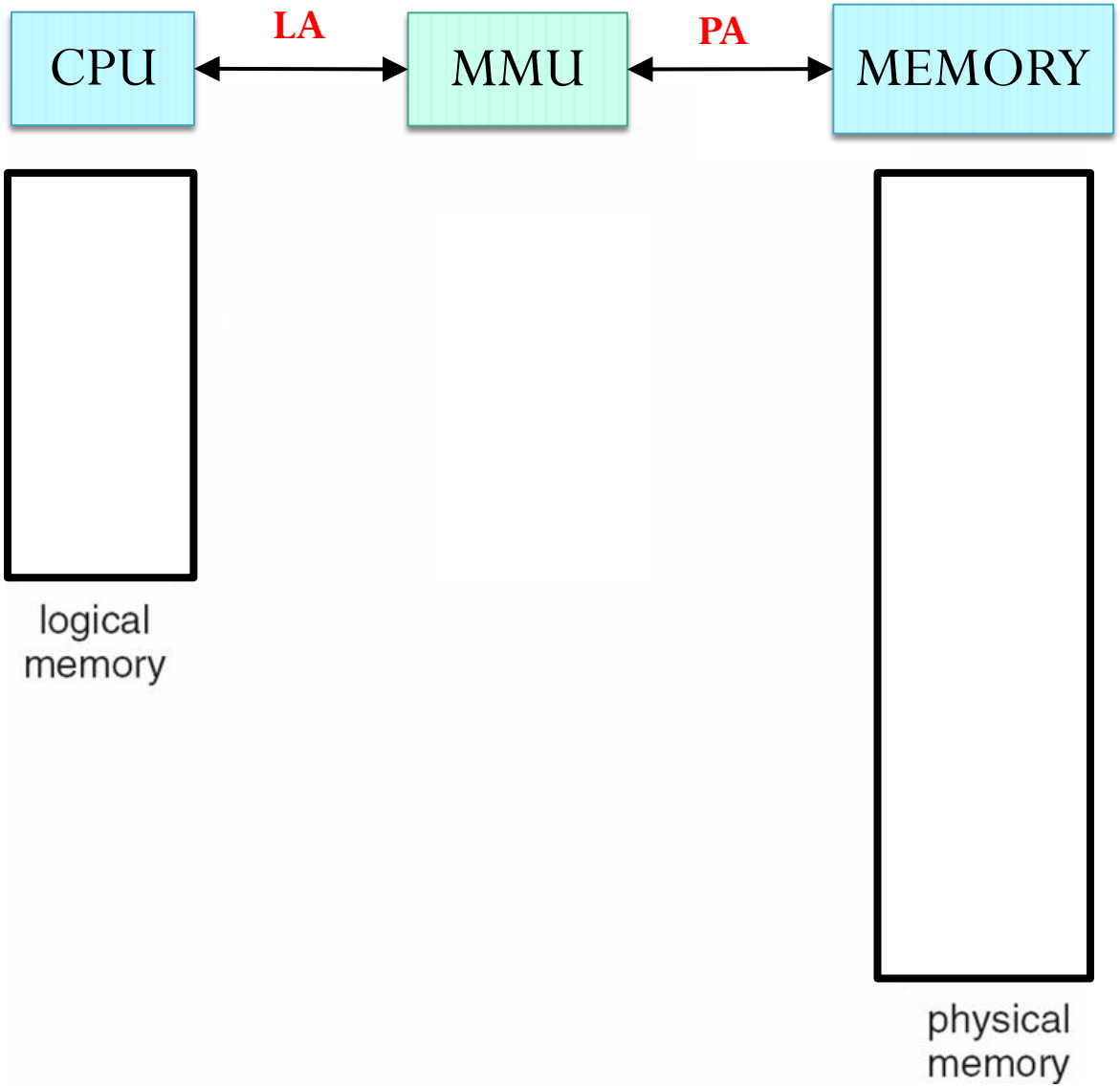
- **Memory-mapped file I/O offers many benefits**
- **Performance benefits**
  - It allows file I/O to be treated as routine memory access (e.g., like an array) by **mapping** a disk block to a page in memory, offering significant performance benefits:
    - A file is initially read using demand paging.
    - Subsequent reads/writes to/from the file are performed through ordinary memory accesses (i.e., using pointers).
      - Using ordinary memory accesses is much cheaper than using **read()/write()** system calls
    - Closing the file results in all the memory-mapped data being written to disk and removed from the virtual memory
- **Easy memory sharing**
  - It also allows several processes to map the same file allowing the pages in memory to be shared

Memory Mapped Files





Paging Model of Logical and Physical Memory



Segmentation

- Memory-management scheme that supports user view of memory
- A program is a collection of segments
  - A segment is a logical unit such as:

    main program

    procedure

    function

    method

    object

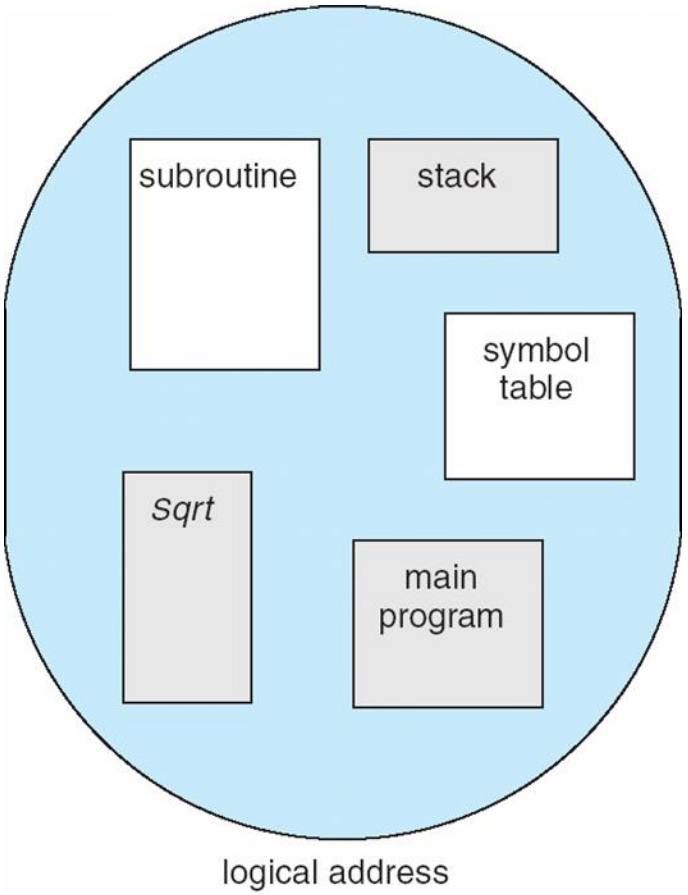
    local variables, global variables

    common block

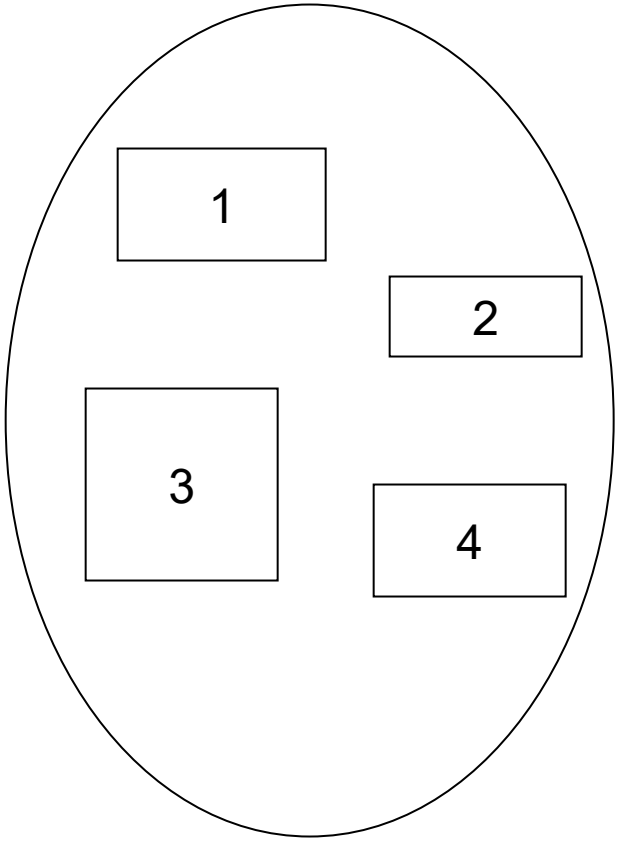
    stack

    symbol table

    arrays



Logical View of Segmentation



user space

physical memory space

Segmentation Architecture

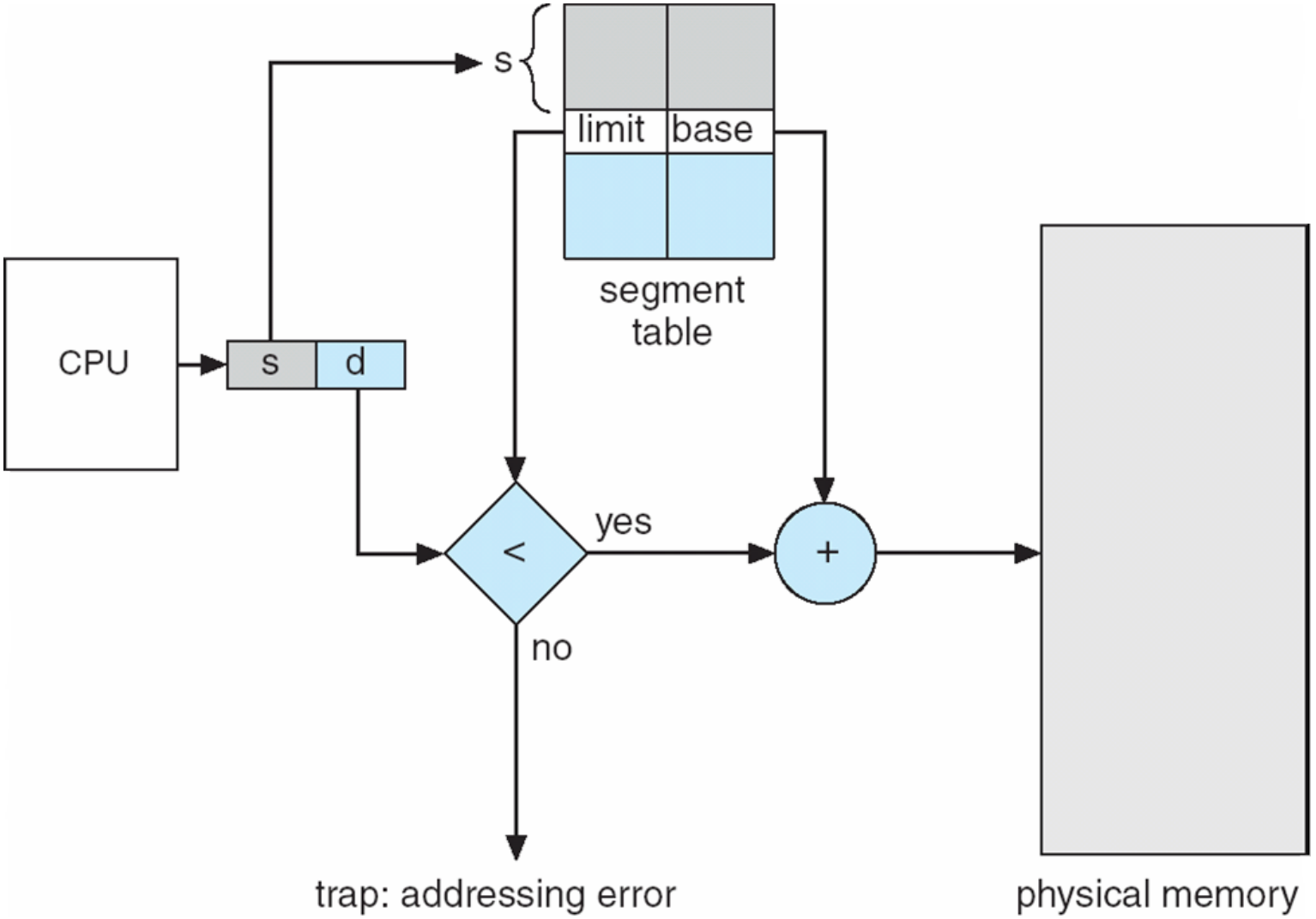
- Logical address consists of a two tuple:

  <**segment-number, offset**>,

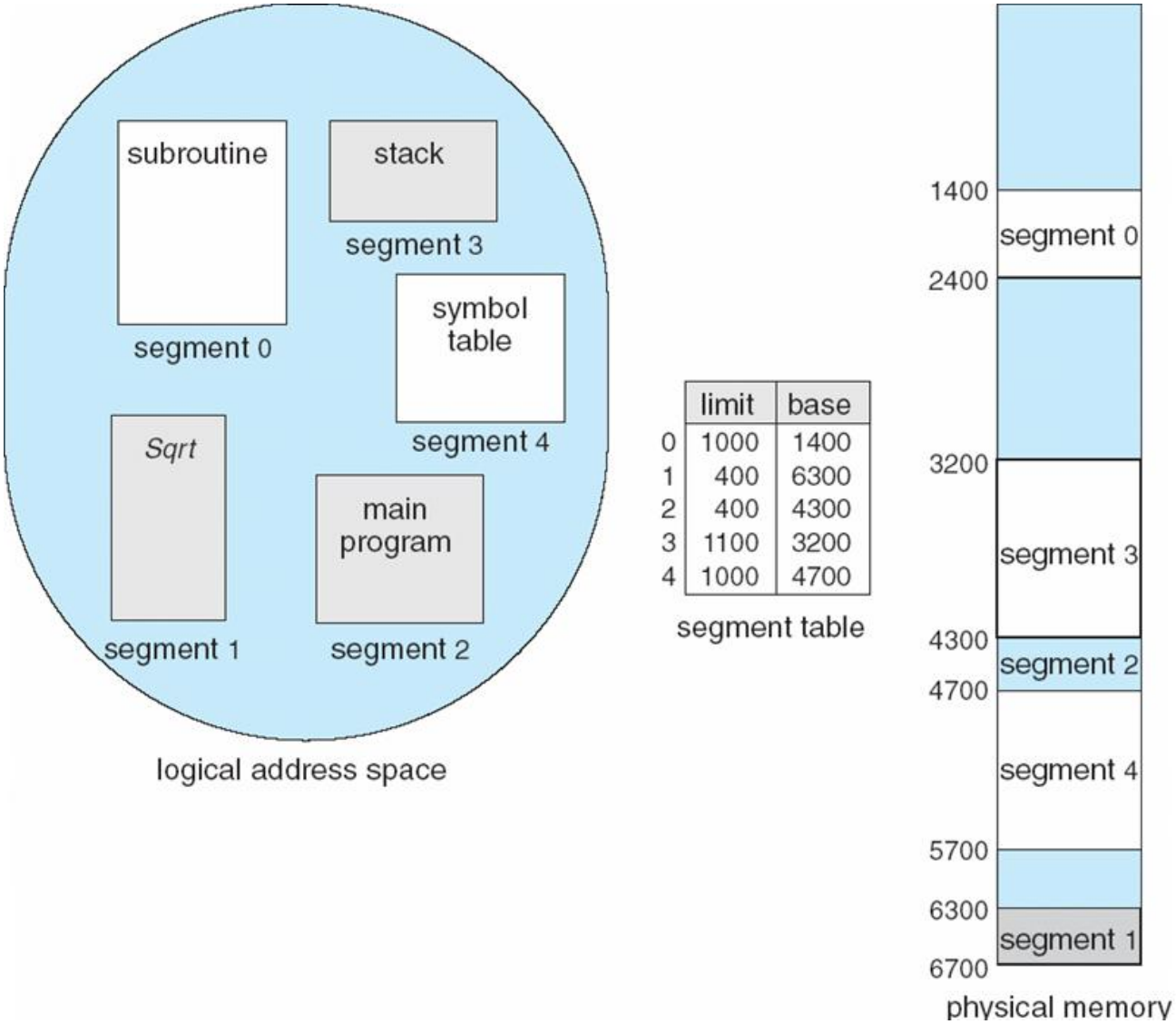
- Segment table – maps two-dimensional physical addresses; each table entry has:
  - base – contains the starting physical address where the segments reside in memory
  - limit – specifies the length of the segment
- Segment-table base register (STBR) points to the segment table's location in memory
- Segment-table length register (STLR) indicates number of segments used by a program; segment number *s* is legal

  if *s* < STLR

Segmentation Hardware

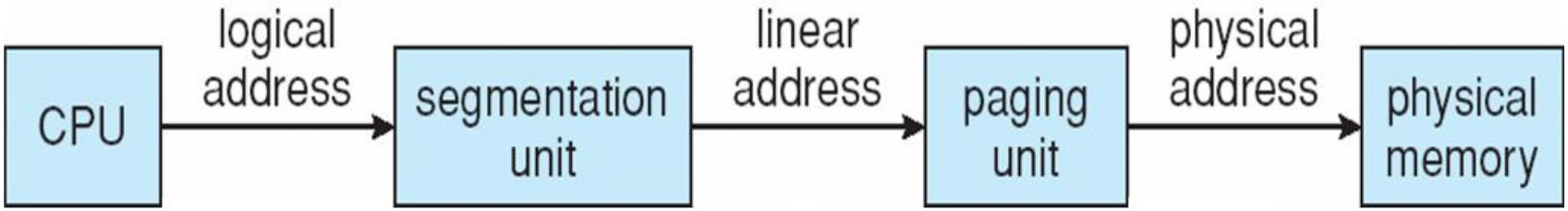


Example of Segmentation



Example: The Intel Pentium

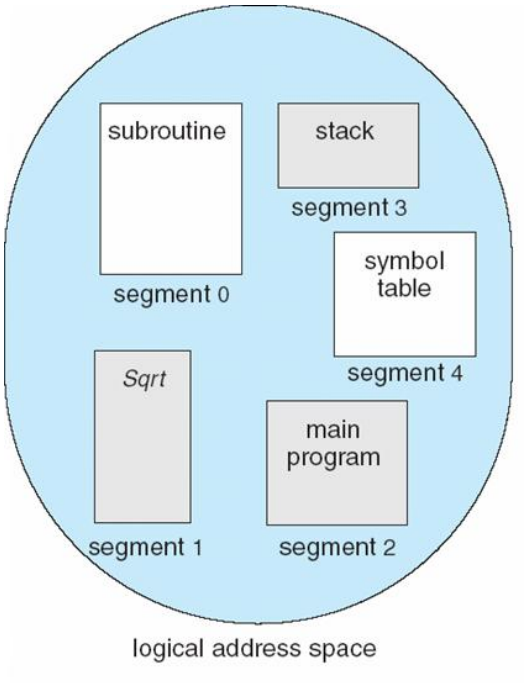
- Supports both segmentation and segmentation with paging
- CPU generates logical address
  - Given to segmentation unit
    - which produces linear addresses
  - Linear address given to paging unit
    - which generates physical address in main memory
    - paging units form equivalent of MMU

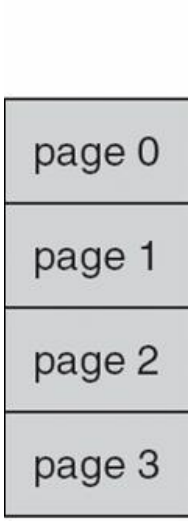


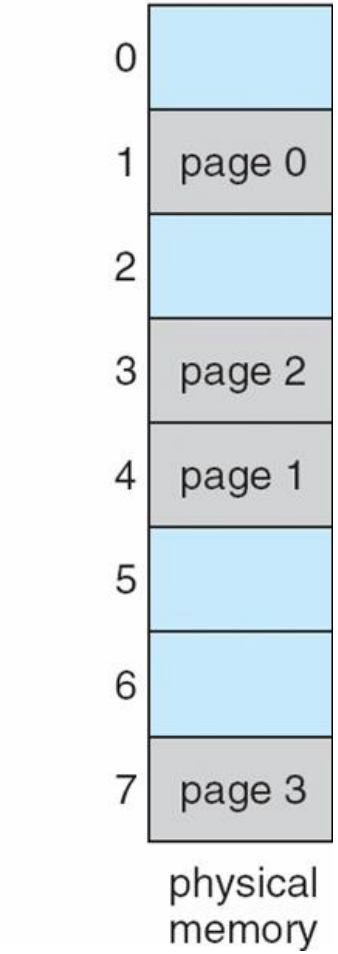
Logical address

Linear address

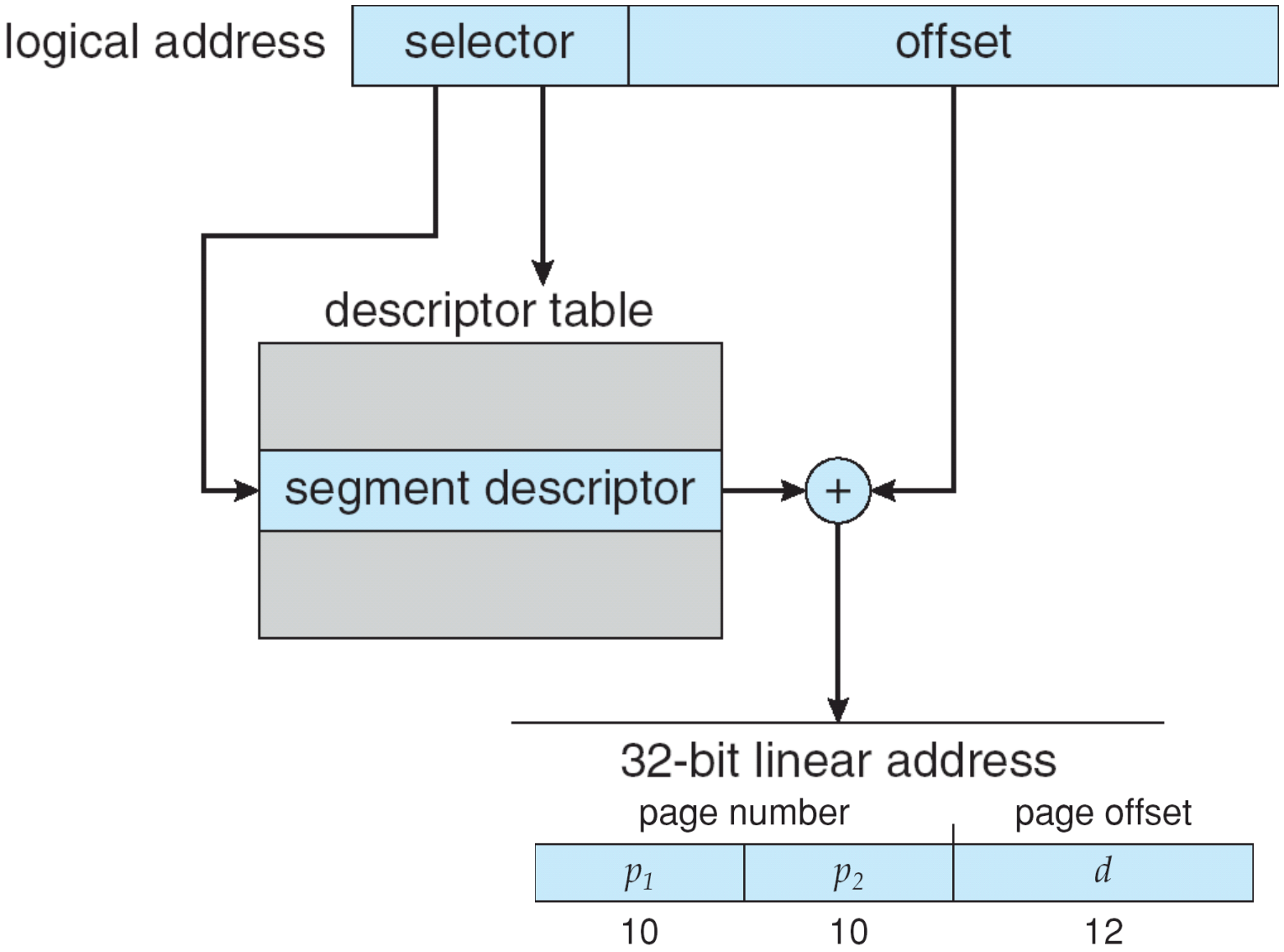
Physical address







Intel Pentium Segmentation



Pentium Paging Architecture